# Device Identification and Anomaly Detection in IoT Environments

Mahdi Rabbani, Jinkun Gui, Fatemeh Nejati, Zeming Zhou, Arun Kaniyamattam, Mansur Mirani, Gunjan Piya, Igor Opushnyev, Rongxing Lu, *Fellow, IEEE*, and Ali A. Ghorbani, *Senior Member, IEEE*

***Abstract*—As the Internet of Things (IoT) landscape continues to expand, a diverse range of devices with various functionalities is being integrated into the IoT ecosystem. When traditional systems, which involve human interaction, are replaced by devices, it becomes crucial to upgrade the conventional authorization and authentication mechanisms. Traditional approaches for device identification and anomaly detection often fail to address the dynamic behaviors of IoT devices due to the highly heterogeneous nature of the IoT environment. To address these challenges, this article proposes a novel and lightweight integrated model for simultaneous IoT device identification and anomaly detection. The proposed approach leverages both packet-based and flow-based feature extraction techniques to extract a diverse and significant set of features, which are crucial for robust anomaly detection and device classification. This novel combined feature set incorporates a wide range of attributes from various domains, including HTTPS-related features, handshake information, and user agent strings, specifically extracted for IoT device identification. In addition, the feature set includes specialized attributes for anomaly detection, such as stream, channel, and jitter metrics, which are calculated over different time intervals to enhance the model's anomaly detection capabilities. Experimental analysis, conducted using real network traffic data from state-of-the-art datasets, demonstrates the model's efficiency and scalability, which makes the model well-suited for real-time IoT threat detection and device management in resource-constrained environments.**

***Index Terms*—Anomaly detection, device classification, Internet of Things (IoT) behavioral analysis, IoT device identification, machine learning.**

## I. Introduction

THE Internet of Things (IoT) is a network of interconnected devices that are capable of interacting with the physical world and communicating with the digital world. An IoT device is one that has at least one transducer capability (sensing or actuating) to interact with the physical environment and one network capability (wired or wireless) to communicate with other devices and servers. These devices can exchange various types of information, including text, voice, video, and sensory data, using specific protocols like Zigbee and LoRaWAN in real-time applications, such as monitoring and automation [1].

In typical IoT scenarios, devices gather data from their environment and transmit it to cloud servers for analysis, where actionable insights are generated. Through Internet connectivity, users can remotely interact with these devices via mobile applications, allowing them to monitor the data and send commands to modify environmental conditions. For example, a user can configure use cases in the Alexa app to control devices like door locks and light bulbs. Commands issued to Alexa are analyzed on cloud servers, triggering appropriate device actions, such as turning on a light bulb [2]. IoT has become an integral part of daily life, with applications across smart homes, smart cities, transportation, healthcare, and manufacturing. The number of IoT devices, technologies, protocols, and users has grown significantly in recent years, and various trends and reports suggest that this expansion will persist [3]. While IoT offers increased convenience and profitability for businesses, it also raises significant security and privacy concerns [4].

A significant challenge arises from the need to balance cost efficiency with security. IoT device manufacturers often prioritize minimizing production costs, resulting in devices with limited processing power and inadequate security mechanisms [5]. Moreover, the absence of adequate compliance and standardization leads to issues of heterogeneity and interoperability, which further exacerbate vulnerabilities. These weaknesses make IoT devices susceptible to becoming entry points for cyberattacks, such as credential tweaking, where inadequate authentication mechanisms enable attackers to compromise devices, form botnets, and launch Distributed Denial of Service (DDoS) attacks [6].

The lack of robust security measures, combined with the increasing heterogeneity and complexity of IoT ecosystems, poses significant risks to users and networks. While extensive research has been conducted in this domain, there is still an urgent need for efficient and scalable solutions to address these vulnerabilities, particularly in detecting and mitigating advanced threats, such as anomaly-based attacks, device misidentification, and zero-day exploits. To address these challenges, this article proposes a novel integrated machine learning model for simultaneous IoT device identification and

Received 7 November 2024; revised 6 December 2024; accepted 19 December 2024. Date of publication 25 December 2024; date of current version 9 May 2025. This work was supported by the Canadian Institute for Cybersecurity (CIC). *(Corresponding author: Mahdi Rabbani.)*

Mahdi Rabbani, Jinkun Gui, Fatemeh Nejati, Zeming Zhou, Arun Kaniyamattam, Rongxing Lu, and Ali A. Ghorbani are with the Canadian Institute for Cybersecurity, Faculty of Computer Science, University of New Brunswick, Fredericton, NB E3B 5A3, Canada (e-mail: m.rabbani@unb.ca; jinkun.gui@unb.ca; fatemeh.nejati@unb.ca; zeming.zhou@unb.ca; arun.kaniyamattam@unb.ca, rlu1@unb.ca; ghorbani@unb.ca).

Mansur Mirani, Gunjan Piya, and Igor Opushnyev are with Mastercard Vancouver Tech Hub, Vancouver, BC V6C 2B3, Canada (e-mail: mansur.mirani@mastercard.com; sunny.piya@mastercard.com; igor.opushnyev@mastercard.com).

Digital Object Identifier 10.1109/JIOT.2024.3522863

2327-4662 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

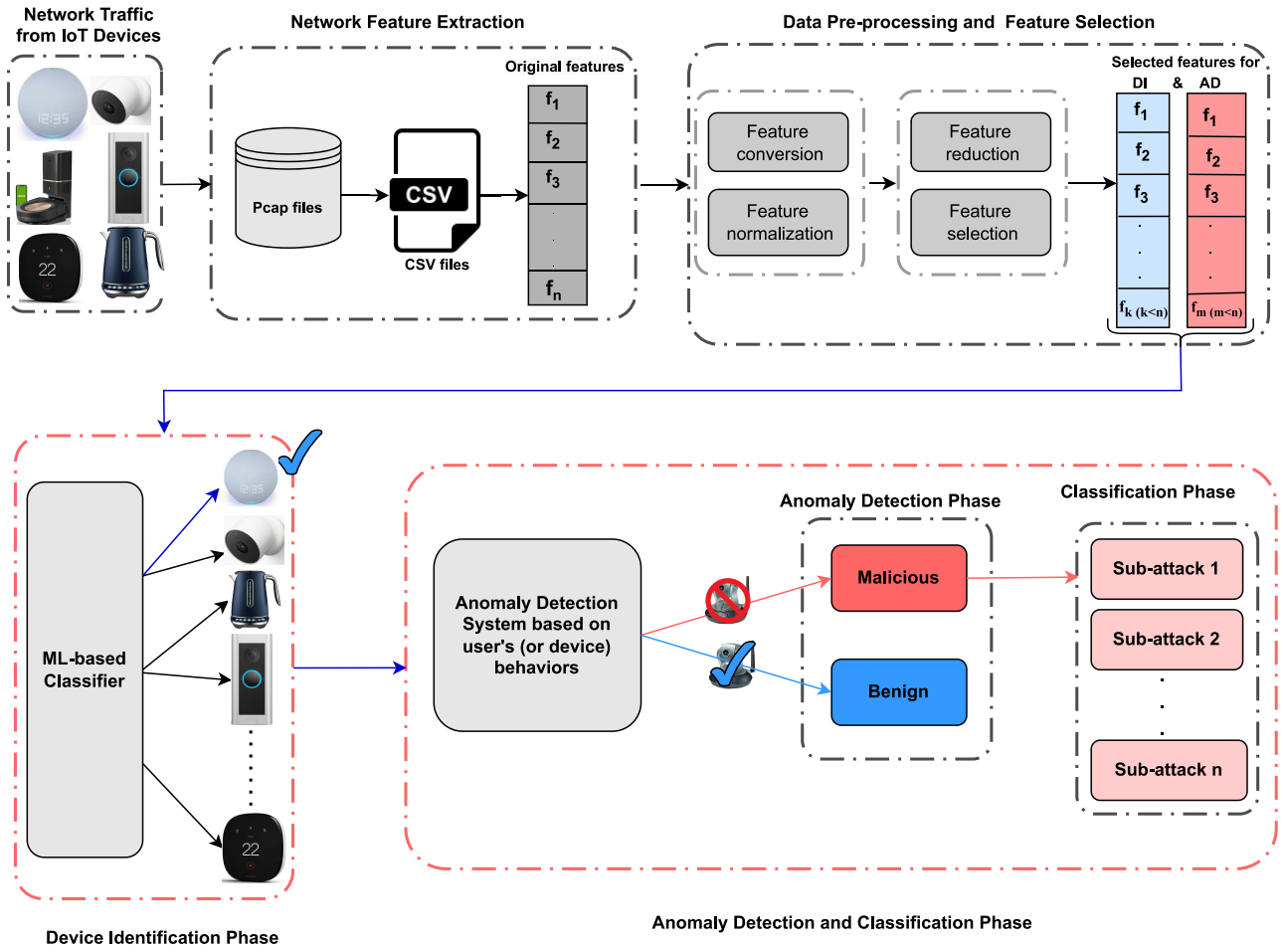



Fig. 1. Workflow of the integrated framework for IoT device identification and anomaly detection system.

anomaly detection. The model leverages a combination of packet-based and flow-based features to create a comprehensive feature set. This set includes diverse attributes from multiple domains, such as HTTPS-related features and user agent strings, specifically extracted for device identification. Furthermore, it incorporates specialized metrics for anomaly detection, including stream, channel, and jitter measurements, calculated over various time intervals to enhance detection accuracy and robustness.

### A. Anomalies in IoT Devices

In IoT devices, anomalies refer to unusual patterns or deviations from normal behavior, which indicate potential issues, such as device malfunctions, security breaches, or environmental changes. These anomalies can vary from simple IoT device malfunctions to advanced cyber-attacks. Detecting these anomalies plays an important role in maintaining the reliability and security of IoT systems. Additionally, understanding these anomalies is critical, because they are often signs of more specific threats to the system. Fig. 1 illustrates the proposed workflow, consisting of two primary phases: 1) IoT device identification to classify different types of IoT devices and 2) anomaly detection to identify anomalies and categorize various types of attacks within IoT environments.

Attacks on IoT systems can take various forms, including physical attacks, network attacks, software attacks, and side-channel attacks [7]. In this article, the focus is on network attacks based on the CICIoT2023 dataset [8], a state-of-the-art dataset recently released in 2023. This dataset includes 33 different attacks across 7 categories, gathered using 105 IoT devices at the Canadian Institute for Cybersecurity. The list of network attacks in the CICIoT2023 dataset as follows.

1) *Denial of Service (DoS) Attack:* A DoS attack occurs when an attacker overwhelms servers by flooding their resources with network packets to prevent users from accessing online services and websites.
2) *DDoS Attack:* A DDoS attack is a type of DoS attack in which an adversary floods servers using a distributed network of compromised devices.
3) *Reconnaissance Attack:* This attack involves gathering information about IoT devices and networks, with the goal of identifying potential vulnerabilities and weaknesses for exploitation.
4) *Web-Based Attack:* This attack includes malicious activities that target Web applications by exploiting vulnerabilities in their design or implementation.
5) *Brute Force Attack:* In this attack, an attacker systematically attempts all possible combinations of passwords or encryption keys until the correct one is discovered.

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

6) *Spoofing Attack:* In this type of attack, an individual or entity pretends to be something else to gain a victim's trust, access a system, steal data, or spread malware. Different forms of these attacks are as follows.
   a) *Arp spoofing:* A hacker sends fake ARP packets that link the attacker's MAC address to the IP address of a legitimate computer that is already present on the local area network (LAN).
   b) *DNS spoofing:* This attack involves manipulating DNS records to redirect users toward a fraudulent, malicious website that may resemble the user's intended destination.
7) *Mirai Attack:* Mirai malware infects IoT devices, transforming them into bots that can be remotely controlled by the attacker. The attacker can then coordinate these infected devices to launch massive DDoS attacks against specific targets, overwhelming them with excessive traffic and rendering their services unavailable to legitimate users.

### B. Main Contributions

This work introduces key contributions that improve both device identification and anomaly detection capabilities, summarized as follows:

1) To the best of our knowledge, this work introduces a novel integrated model that simultaneously addresses both IoT device identification and anomaly detection.
2) We extracted behavioral-based features using two complementary methods: a) a packet-based approach and b) a flow-based approach to obtain a diverse set of features essential for robust anomaly detection and device identification.
3) A subset of the extracted features was selected specifically for IoT device identification, incorporating domains, such as stream data, HTTPS-related features, handshake information, and user-agent strings. This feature set was then used to develop a lightweight device identification model leveraging device behavior characteristics, which demonstrated high effectiveness in accurately classifying IoT devices.
4) The feature set was expanded to include behavioral characteristics crucial for distinguishing malicious activity from benign device behavior. The anomaly detection model was upgraded with these features, including stream, channel, and jitter metrics calculated over various time intervals to strengthen its anomaly detection capabilities.
5) The experimental analyses were conducted using real network traffic data (packet capture (PCAP) files) from the state-of-the-art CICIoT2023 attack dataset. The proposed dual-functionality model demonstrated promising performance in both tasks, which emphasize the potential for deploying efficient, scalable, and lightweight models capable of performing anomaly detection and device identification in resource-constrained IoT environments.

The remainder of this article is organized as follows. Section II provides an overview of existing studies related to various techniques for anomaly detection in IoT devices. In Section III, we discuss the details of the proposed feature extraction techniques, preprocessing steps, and model training. Section IV presents the outcomes of different experimental analyses based on different feature extraction perspectives and their corresponding results. Finally, Section V brings this article to a conclusion, offering our final reflections and remarks.

## II. Related Work

This section reviews recent research efforts in anomaly detection for IoT devices, with a focus on various methodologies, techniques, and their applications. The objective is to evaluate these approaches, highlighting their advantages and disadvantages. Specifically, the review emphasizes machine learning-based approaches for anomaly detection presented in the literature.

### A. Supervised Learning Techniques

Kraiem et al. [9] introduced the composition-based decision tree (CDT), a model designed for anomaly detection in univariate time series data. The CDT generates a minimal set of interpretable rules which addresses the need for explainable machine learning in IoT environments. Additionally, the authors employ Bayesian optimization to automate hyperparameter tuning to enhance the model's computational efficiency. While the CDT demonstrates interpretability, its focus on univariate data limits its applicability to more complex, multivariate scenarios typical in IoT systems. In contrast, our work presents an integrated system that combines both device identification and anomaly detection to enhance authentication processes.

Evmorfos et al. [10] presented a random neural networks (RNNs) for detection of TCP SYN attacks. It offers an alternative lightweight approach that excels in handling complex patterns in nonlinear data. RNNs demonstrated higher accuracy (80.7%) and a lower false positive (FP) rate (19.3%) than LSTM in SYN flood attack detection, although their ability to capture temporal dependencies is less robust. Chen et al. [11] proposed a framework called graph transformer anomaly detection (GTA), combined with Transformer architectures, which represents another promising direction. These models dynamically learn relationships between IoT sensors using Gumbel-Softmax sampling and enhance anomaly detection through multibranch attention mechanisms. With an F1 score of 0.91 on the SWaT dataset, this approach shows exceptional performance. However, the computational demands of graph convolution and Transformers are best handled by high-performance computing environments.

Abbasi et al. [12] focused on logistic regression (LR) and artificial neural networks (ANNs) for efficient anomaly detection. These methods offer simplicity and speed, achieving over 98% accuracy in binary classification on the N-BAIOT dataset. While LR models are lightweight and easy to train, they struggle to capture nonlinear relationships. ANN improves performance but requires more computational resources than LR, limiting its scalability. Yaras and Dener [13] proposed

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

convolutional neural network (CNN)-LSTM hybrid models, which utilize convolutional layers to capture spatial features and LSTM layers to learn temporal dependencies. These models have demonstrated exceptional performance, achieving up to 99.96% accuracy on multiclass and binary classification tasks. However, their high computational complexity requires substantial resources, making it hard to achieve real-time detection.

In addition to these methods, Khan and Alkhathami [14] used denoising autoencoders (DAs) to enhance robustness by extracting key features from noisy data. While these models perform well in complex scenarios, they also raise high computational costs making them only suitable primarily for cloud-based deployment. Abusitta et al. [15] proposed a deep-learning-based anomaly detection in IoT that is capable of extracting useful and effective features that a classification model uses to improve the detection accuracy of abnormal network traffic and demonstrating flexibility and effectiveness in different computing environments. Although their design simplifies deployment, they require careful parameter tuning to maintain optimal real-time performance.

### B. Unsupervised Learning Techniques

Liu et al. [16] introduced an efficient unsupervised deep learning model designed to balance performance and efficiency for anomaly detection over edge-assisted IoT networks. Their model utilizes a convolutional autoencoder with adversarial training, accompanied by an adaptive strategy to reduce runtime overhead for continuous monitoring. The adversarial training improves detection accuracy by learning more representative features, and the adaptive strategy ensures low latency, making it suitable for real-time applications. However, the complexity introduced by adversarial training may increase the computational load during model updates, which might limit its deployment on resource-constrained edge devices. Apostol et al. [17] developed an intrusion detection system (IDS) based on an unsupervised deep autoencoder aimed at detecting IoT botnets. They evaluate the model on both balanced and unbalanced datasets to assess its robustness, and they adjust thresholds to reduce FPs. This approach demonstrates strong performance in detecting botnets, even under challenging conditions with imbalanced data. However, the reliance on threshold adjustments adds complexity to deployment, as fine-tuning these thresholds for different environments may require significant domain expertise.

Hwang et al. [18] presented D-PACK, an anomaly detection model for IoT network traffic that combines a CNN with an unsupervised deep autoencoder. D-PACK is highly efficient in early-stage detection, reducing response times for mitigating threats. Its reliance on a limited amount of data makes it lightweight and suitable for real-time IoT environments. However, the limited packet sampling could reduce its ability to detect complex, long-duration attacks, potentially affecting detection accuracy in more sophisticated scenarios. Guo et al. [19] proposed an anomaly detection framework targeting multivariate time-series data in IoT systems for smart cities. They employ a gated recurrent unit (GRU) to capture temporal correlations and a Gaussian mixture model (GMM) to characterize multimodal data. To address the limitations of standard GMMs, they integrate a model selection mechanism based on the Bayesian inference criterion (BIC) to optimize the distribution estimation. This approach excels in detecting anomalies in multimodal data by balancing complexity and accuracy, particularly in dynamic environments. However, the model selection process increases computational overhead. Additionally, the performance may degrade if the temporal correlations are weak or inconsistent across data points.

Bhatia et al. [20] introduced a behavior-centric, network-based unsupervised machine learning model for anomaly detection in vulnerable IoT environments. Their system utilizes an autoencoder trained on normal traffic to detect anomalies and DDoS attacks in industrial networks. The model's focus on leveraging TCP traffic patterns enhances its ability to detect a variety of DDoS attacks, and it performs well in industrial IoT networks with predictable traffic behavior. However, the model's reliance on predictable traffic patterns limits its applicability to more diverse IoT networks, where traffic behaviors are highly dynamic and unpredictable.

### C. Semi-Supervised Learning Techniques

Lutsiv et al. [21] proposed an IDS using a deep recurrent autoencoder to detect anomalies in heterogeneous IoT environments. The key strength of their approach lies in the 94% reduction in feature dimensions without compromising performance, which enhances computational efficiency. However, the model may struggle with unseen attacks and is computationally intensive during the training phase, limiting its scalability in real-time IoT systems. Nukavarapu and Nadeem [22] introduced a semi-supervised GAN-based multiclass classifier, capable of device identification and zero-day attack (ZDA) detection. Its ability to operate on minimal labeled data makes it suitable for dynamic and evolving IoT systems. However, GANs are known for their unstable training process, and the computational cost may pose challenges in resource-constrained edge environments.

Liu et al. [23] proposed a dynamic ensemble algorithm combining semi-supervised extreme learning machines (SSELMs) with mutual information criteria. The model's simple structure and fast training make it ideal for lightweight IoT devices. However, its reliance on predefined base classifiers can limit flexibility in adapting to new attack patterns, and its performance may degrade with noisy data. Cheng et al. [24] addressed the label scarcity in IoT networks by developing a hierarchical stacking temporal convolutional network (HS-TCN). The model captures the streaming characteristics of IoT data, using semi-supervised learning to label unlabeled data incrementally. However, the hierarchical stacking process increases the model's complexity, requiring significant computational resources during both training and inference, which may limit its deployment in real-time scenarios.

Abdel-Basset et al. [25] developed SS-Deep-ID, which uses a multiscale residual temporal convolutional module to improve spatiotemporal representation learning and an attention mechanism to focus on key features. The model

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

shows high detection accuracy even with limited labeled data. However, the reliance on multiple modules, including attention mechanisms, increases the computational burden, posing challenges for real-time deployment in IoT environments with limited resources. Meyer-Berg et al. [26] designed a framework to generate synthetic IoT datasets for anomaly detection. This approach enables model evaluation in a controlled environment, helping to simulate diverse attacks and network conditions. However, the simulated data may not fully capture the complexities of real-world traffic, potentially impacting the generalizability of the trained models.

Overall, the main limitations of the existing works are summarized in the following.

1) Unsupervised learning methods, while effective in reducing the need for labeled data, often lack interpretability, making it difficult for practitioners to understand the decision-making process.
2) Many high-accuracy schemes are computationally intensive, limiting their applicability. Some methods depend heavily on high-performance hardware platforms, which restricts their use in real-time or edge environments.
3) Several studies rely on pre-existing features provided by dataset creators, which may not capture the full complexity of IoT environments or adapt to evolving threat patterns.

In conclusion, this section has provided an overview of recent advancements in IoT device anomaly detection. While each method offers unique strengths, it also exhibits key limitations in interpretability, scalability, and adaptability. Our proposed model addresses these gaps by integrating both device identification and behavioral-based features to improve detection accuracy, offering a scalable solution for real-world IoT environments.

## III. Feature Extraction and Model Training

This section explains the feature extraction process for anomaly detection in IoT devices. It begins with essential background information, followed by a description of two different approaches used to extract significant behavioral-based features. Finally, the extracted features are presented for the training phase of the anomaly detection model. Feature extraction in network traffic analysis is a critical process that involves isolating specific characteristics from network data. These characteristics (features) are crucial for further analysis, particularly in detecting anomalies or identifying the types of devices communicating over the network. In order to extract a comprehensive feature set from different dimensions, we employed two approaches.

1) *Packet-Based Approach:* This approach focuses on analyzing each individual packet that travels through a network to capture detailed information about the network's behavior and security state. By examining packet-level data, we can gain insights into the specific actions and communications of devices, which allows the detection of minute irregularities that may indicate security breaches or malfunctioning devices.
2) *Flow-Based Approach:* In this approach, we extract features from a flow in a network, which includes the entire conversation between two devices. This flow is characterized by five key components: *Source IP*, *Destination IP*, *Source Port*, *Destination Port*, and *Protocol*. This approach provides a broader view of network traffic by considering the full communication session between devices. It is particularly useful for identifying patterns that might be missed at the packet level, such as persistent connections or repeated attempts to access specific resources.

### A. Feature Extraction Using Packet-Based Approach

This section focuses on the packet-based approach for feature extraction in the anomaly detection phase. This method is particularly important in our problem domain, as it provides a powerful mechanism for network traffic analysis, allowing for detailed insights into each packet's role and behavior within the network. By examining packet-level features, network administrators and security experts can identify potential security threats and gain a better understanding of network operations. This detailed approach is essential for maintaining secure and reliable networks, especially in complex environments, such as IoT systems, where security and performance are critical.

*1) Network Packets and Data Communication:* Network packets serve as the fundamental units of communication within a network, encapsulating data for transmission. They play a critical role in network functionality, as they carry the necessary information for data transfer, including both the content itself and control information to ensure accurate and efficient delivery. The relationship between network packets and networks is fundamental, as the design, operation, and management of networks are entirely centered on the transmission and processing of packets. Each packet consists of two main components: 1) header information and 2) a payload, as described as follows.

1) *Header:* The header of a packet includes essential control information, such as source and destination IP addresses, port numbers, protocol type, and other relevant data. This information directs the packet to its destination and ensures correct processing.
2) *Payload:* The payload refers to the actual data carried by the packet. This section can vary significantly depending on the application, ranging from simple text to complex multimedia content. The payload represents the essential content that needs to be delivered from one network device to another.

Packets are responsible for encapsulating data within the network. Data encapsulation is a fundamental process in network communication, where higher-level application data (such as HTTP requests, email content, etc.) is packaged into a format suitable for transmission. This involves layering, where data moves down the protocol stack from the application layer, with each layer adding relevant header information for its specific function. The packet is then routed based on this encapsulated header information. Routing involves determining the packet's path from the source to the destination. At each network node, such as a router, the packet's header is examined (particularly the destination IP address) and the node

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

uses the routing table to decide the packet's next hop toward its final destination. This process is repeated until the packet reaches its target, which guarantees accurate and efficient data delivery across the network.

*2) Packet Capturing and Analysis:* Packet capturing is a fundamental technique in network management and security analysis, essential for capturing and examining network packets to understand network behavior, identify issues, and enhance security. This technique is widely used in network performance monitoring, troubleshooting, security investigations, and ensuring compliance of network applications. Packet capturing is typically performed by specialized software that can listen to network interfaces and record all packets passing through that interface. Wireshark and tcpdump are among the most commonly used tools, which offer user-friendly interfaces for both capturing and analyzing network traffic.

Captured packet data are typically stored in a PCAP file, a widely accepted format compatible with various network analysis tools. The PCAP file format preserves information about network packets and is supported by numerous packet capturing and network analysis tools, such as Wireshark, tcpdump, etc. PCAP files enable users to save detailed captured packet information for later analysis or to serve as evidence in network diagnostics and security investigations.

*3) PCAP to CSV Conversion for Enhanced Analysis:* In our laboratory traffic analysis, raw network traffic data are stored in the PCAP file format, recognized for its comprehensive information about network packets. However, fully converting all PCAP data into CSV format presents challenges due to the volume and complexity of data. To address this, we focused on extracting key behavioral features essential for anomaly detection and device identification in IoT devices. The PCAP2CSV tool enables efficient extraction of selected features from PCAP files and stores them in CSV format. This approach preserves critical data for analysis and significantly enhances the efficiency and effectiveness of network traffic analysis.

*4) Experimental Setting:* This section describes the tools, libraries, and methods used to implement and evaluate our system. Our experimental setup utilizes various Python packages and statistical tools designed to facilitate detailed packet analysis and efficient data processing.

1) *Python Package (Packet Reading by Pyshark):* Pyshark [27] is a Python wrapper for *tshark*, enabling packet parsing via Wireshark dissectors. Unlike other parsing modules, Pyshark does not parse packets directly. This approach ensures accurate and detailed packet analysis by utilizing Wireshark's extensive dissector capabilities. Pyshark supports packet parsing from both capture files and live captures, using all installed Wireshark dissectors. It is compatible with both Windows and Linux, and its robust decoding functionality requires no additional setup, even for complex protocol features like TLS decryption.
2) *Statistical Tools:* These include `statistics` for basic operations, `numpy` for numerical computations, and `math.log` for calculating natural and base logarithms. These tools are essential for conducting in-depth statistical analysis on network traffic data, which provides insights into patterns and anomalies. Additionally, `collections. Counter`, a subclass of the Python `collections` module, counts and tracks elements efficiently, which is particularly useful for network traffic analyses.
3) *Speed Enhancement Through Multiprocessing:* Multiprocessing employs multiple processors on a machine to significantly accelerate computation-intensive operations through parallel processing. This approach is particularly beneficial for tasks involving large-scale data processing or requiring real-time analysis, as it ensures that the system remains responsive and efficient.

*5) Our Approach for the PCAP2CSV Process:* For packet-by-packet analysis and feature extraction, our methodology consists of two primary steps. First, we employ Pyshark to read each packet in a PCAP file sequentially, recording key data or information requiring further analysis that can be decoded directly. For instance, we can obtain the target port number of a TCP packet using the following command:

```
src_port \ensuremath{=} curr_packet.tcp.srcport.
```

Since no additional processing is necessary, these features refer to *readable features*. These features represent directly accessible and immediately usable data points extracted from the packets. Subsequently, we perform further statistical analysis on these readable features to improve the distinction between normal and abnormal behaviors. This additional computation results in what we refer to as *calculation features*, which are derived features created by applying statistical methods to the raw packet data and provide deeper insights into network behavior.

For the IoT device identification phase, we assume all devices are benign, meaning they are not under attack or launching attacks against other devices. Additionally, the functions or communication targets of IoT devices tend to be more limited. As a result, we can map communication targets (which assist in determining the device's manufacturer and functionality) and specific performance characteristics (such as the types of ciphers and window sizes accepted in TLS handshakes, which tend to be consistent across the same device) as readable features for device identification. In this context, calculation features play a more supportive role.

However, in anomaly detection, the scenario changes significantly. When a device is under a brute force or port scanning attack, it may engage in multiple one-off communications with various targets. In these cases, the relatively static readable features may not provide sufficient differentiation to distinguish between benign and abnormal traffic. During such attacks, a noticeable increase in the number of packets communicated within a specific timeframe often occurs. For instance, a brute force attack generates a substantial volume of traffic directed at the same port, whereas a port scan attack results in a high number of small packets. In these situations, incorporating additional *calculation features* improves

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

TABLE I
AVERAGE OF TOTAL PACKET IN THE SAME STREAM AND CHANNEL AND SUM OF JITTER OF THE SAME STREAM IN 30 S

| Traffic Type | Stream | Channel | Jitter |
|---|---|---|---|
| Benign Traffic | 97.7 | 234.5 | 315.4 |
| Dictionary Brute Force | 93.5 | 352.9 | 1416.3 |
| DDoS-ACK_Fragmentation | 10228.5 | 5834.6 | 29.9 |
| DDoS-ICMP_Fragmentation | 35904.5 | 5299.3 | 29.3 |
| DDoS-SlowLoris | 229.6 | 719.6 | 14.6 |
| DDoS-SynonymousIP_Flood | 3.3 | 96509.1 | 1.6 |
| DoS-TCP_Flood | 2.1 | 99277.8 | 6.7 |
| DoS-UDP_Flood | 823.5 | 98467.7 | 0.7 |
| MITM-ArpSpoofing | 35607.8 | 35608 | 28.9 |
| Recon-OSScan | 288.6 | 788.2 | 76.2 |
| Recon-PortScan | 22.11 | 1347.7 | 164.9 |

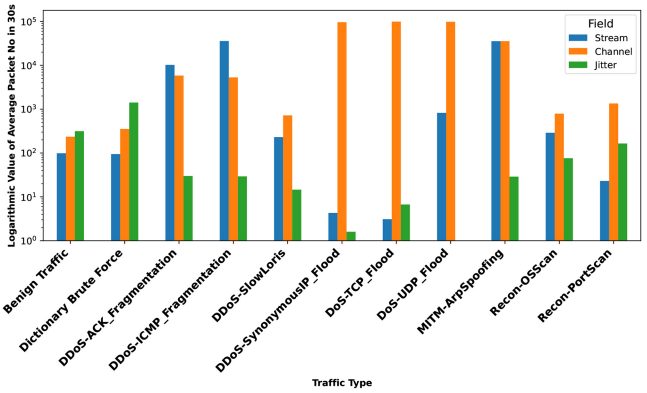



Fig. 2. Visualization of average packets, channel, and jitter for different traffic types over 30 s.

TABLE II
AVERAGE JITTER VALUES IN DIFFERENT TIME INTERVALS

| Traffic Type | 1s | 5s | 30s | 60s |
|---|---|---|---|---|
| Benign Traffic | 264.4 | 298.5 | 315.4 | 331.1 |
| Dictionary Brute Force | 567.6 | 746.9 | 1416.3 | 2136.2 |
| DDoS-ACK_Fragmentation | 4.8 | 8.7 | 29.9 | 49.8 |
| DDoS-ICMP_Fragmentation | 1.4 | 5.4 | 29.3 | 53.1 |
| DDoS-SlowLoris | 5.9 | 8.4 | 14.6 | 19.6 |
| DDoS-SynonymousIP_Flood | 1.0 | 1.1 | 1.6 | 1.6 |
| DoS-TCP_Flood | 6.4 | 6.4 | 6.4 | 6.4 |
| DoS-UDP_Flood | 0.2 | 0.2 | 0.3 | 0.3 |
| MITM-ArpSpoofing | 69.2 | 5.9 | 28.9 | 52.7 |
| Recon-OSScan | 53.4 | 60.5 | 76.2 | 90.3 |
| Recon-PortScan | 130 | 144.2 | 164.9 | 181.9 |

the capability to differentiate between benign and abnormal traffic. The following changes were made regarding calculation features.

1) *Stream:* A stream refers to the flow of data exchanged during a protocol conversation between two endpoints. It is typically defined as a *5-tuple*, consisting of the protocol (i.e., TCP or UDP), source IP address, source port, destination IP address, and destination port. These five data points collectively identify a single stream within a capture and provide a comprehensive view of the ongoing communication.
2) *Jitter:* This metric measures the time variance between packets within the same stream, which can indicate fluctuations in network performance. The method for calculating jitter in Python is illustrated in Fig. 4.
3) *Channel:* This feature focuses exclusively on the communication between two IP addresses, regardless of the protocol or port they use. This approach provides a broader perspective on the interactions between network entities. The method for defining a channel in Python is presented in Fig. 5.
4) *src_ip_mac:* This feature identifies whether an MAC address is associated with multiple IP addresses, which may indicate specific types of network behavior and potential anomalies.

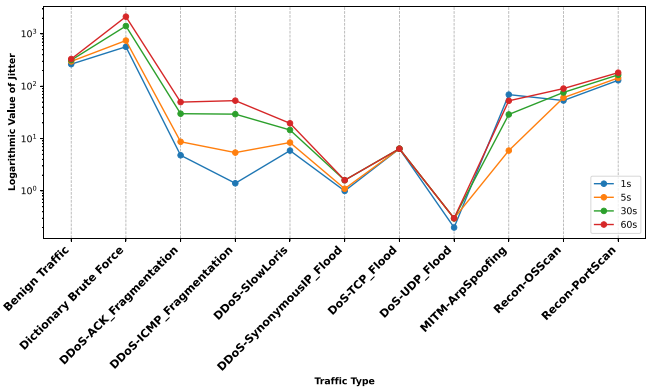



Fig. 3. Visualization of average jitter values across time intervals.

```
df['time_diff'] = df.groupby('stream')['time'].diff()
df['jitter'] = df['time_diff'].dt.total_seconds()
```

Fig. 4. Example of defining jitter in Python.

```
def define_channel(row):
    ip_pair = tuple(sorted([row['src_ip'], row['dst_ip']]))
    return f"{ip_pair[0]}_{ip_pair[1]}"
```

Fig. 5. Example of defining channel in Python.

We then analyze these groups across different time periods, assessing various metrics, such as packet count, mean, variance, median of packet sizes, and jitter. Specifically, sliding time windows are applied for each target to capture data at fixed intervals to record packet sizes and timestamps. Simple statistical operations on these windows enable effective monitoring and analysis of data changes over time. For example:

```
df.groupby(stream)['packet_size'].
transform(lambda x: x.rolling('30s').mean())
```

The above code snippet computes the average size of packets within the same stream over a rolling window of 30 s. This process updates continuously with new incoming data to ensure the analysis reflects the most recent network conditions.

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPLETE SET OF BEHAVIOR-BASED FEATURES EXTRACTED USING PACKET-BASED APPROACH FOR DEVICE IDENTIFICATION AND ANOMALY DETECTION

| No | Feature name | No | Feature name | No | Feature name | No | Feature name |
|---|---|---|---|---|---|---|---|
| 1 | stream | 35 | http_uri | 69 | channel_1_mean | 103 | stream_jitter_10_var |
| 2 | (device_mac) Label 1 for DI | 36 | http_content_len | 70 | channel_1_var | 104 | stream_30_count |
| 3 | src_ip | 37 | http_content_type | 71 | stream_jitter_1_sum | 105 | stream_30_mean |
| 4 | dst_ip | 38 | icmp_type | 72 | stream_jitter_1_mean | 106 | stream_30_var |
| 5 | src_port | 39 | icmp_checksum_status | 73 | stream_jitter_1_var | 107 | src_ip_30_count |
| 6 | dst_port | 40 | icmp_data_size | 74 | stream_5_count | 108 | src_ip_30_mean |
| 7 | inter_arrival_time | 41 | ntp_interval | 75 | stream_5_mean | 109 | src_ip_30_var |
| 8 | time_since_previously displayed_frame | 42 | most_freq_spot | 76 | stream_5_var | 110 | src_ip_mac_30_count |
| 9 | port_class_dst | 43 | min_et | 77 | src_ip_5_count | 111 | src_ip_mac_30_mean |
| 10 | l4_tcp | 44 | q1 | 78 | src_ip_5_mean | 112 | src_ip_mac_30_var |
| 11 | l4_udp | 45 | min_e | 79 | src_ip_5_var | 113 | channel_30_count |
| 12 | ttl | 46 | var_e | 80 | src_ip_mac_5_count | 114 | channel_30_mean |
| 13 | eth_size | 47 | q1_e | 81 | src_ip_mac_5_mean | 115 | channel_30_var |
| 14 | tcp_window_size | 48 | sum_p | 82 | src_ip_mac_5_var | 116 | stream_jitter_30_sum |
| 15 | payload_entropy | 49 | min_p | 83 | channel_5_count | 117 | stream_jitter_30_mean |
| 16 | handshake_version | 50 | max_p | 84 | channel_5_mean | 118 | stream_jitter_30_var |
| 17 | handshake_cipher_suites_length | 51 | med_p | 85 | channel_5_var | 119 | stream_60_count |
| 18 | handshake_cipher_suites | 52 | average_p | 86 | stream_jitter_5_sum | 120 | stream_60_mean |
| 19 | handshake_extensions_length | 53 | var_p | 87 | stream_jitter_5_mean | 121 | stream_60_var |
| 20 | tls_server | 54 | q3_p | 88 | stream_jitter_5_var | 122 | src_ip_60_count |
| 21 | handshake_sig_hash_alg_len | 55 | q1_p | 89 | stream_10_count | 123 | src_ip_60_mean |
| 22 | http_request_method | 56 | iqr_p | 90 | stream_10_mean | 124 | src_ip_60_var |
| 23 | http_host | 57 | l3_ip_dst_count | 91 | stream_10_var | 125 | src_ip_mac_60_count |
| 24 | http_response_code | 58 | jitter | 92 | src_ip_10_count | 126 | src_ip_mac_60_mean |
| 25 | User Agent | 59 | stream_1_count | 93 | src_ip_10_mean | 127 | src_ip_mac_60_var |
| 26 | dns_server | 60 | stream_1_mean | 94 | src_ip_10_var | 128 | channel_60_count |
| 27 | dns_query_type | 61 | stream_1_var | 95 | src_ip_mac_10_count | 129 | channel_60_mean |
| 28 | dns_len_qry | 62 | src_ip_1_count | 96 | src_ip_mac_10_mean | 130 | channel_60_var |
| 29 | dns_interval | 63 | src_ip_1_mean | 97 | src_ip_mac_10_var | 131 | stream_jitter_60_sum |
| 30 | dns_len_ans | 64 | src_ip_1_var | 98 | channel_10_count | 132 | stream_jitter_60_mean |
| 31 | eth_src_oui | 65 | src_ip_mac_1_count | 99 | channel_10_mean | 133 | stream_jitter_60_var |
| 32 | eth_dst_oui | 66 | src_ip_mac_1_mean | 100 | channel_10_var | 134 | Label 2 for AD |
| 33 | payload_length | 67 | src_ip_mac_1_var | 101 | stream_jitter_10_sum | | |
| 34 | highest_layer | 68 | channel_1_count | 102 | stream_jitter_10_mean | | |

The calculated average packet size within this 30-s window is subsequently used as a feature (specifically, feature 76 in Table III) in our anomaly detection model. This feature helps in identifying deviations from normal traffic patterns, which may indicate potential security threats or other issues in the IoT environment.

Table I presents a comparison of normal and abnormal traffic. The Stream column indicates the average number of packets within the same stream over a 30-s interval for different traffic types. For instance, benign traffic averages 97.7 packets, while DDoS-ICMP fragmentation shows an average of 35 904.5 packets. These significant differences in packet counts between benign and malicious traffic types provide a clear indicator for identifying potential security threats and abnormal traffic patterns. A visualization of these differences is illustrated in Fig. 2. Regarding the Channel feature, which is another critical aspect of anomaly detection in IoT devices, as indicated in the third column of Table I, this feature presents the average number of packets within the same channel over 30 s for various traffic types. In this case, benign traffic averages 234.5 packets, while DoS-TCP Flood demonstrates an exceptionally high average of 99 277.8 packets. The disparity in packet volume per channel serves as another crucial metric for differentiating between normal and abnormal communication patterns.

Additionally, in Table I, the last column, representing the Jitter feature, shows the average sum of jitter for various traffic types over a 30-s interval. For example, benign traffic exhibits an average jitter of 315.4, whereas Dictionary Brute Force attacks display a significantly higher average jitter of 1416.3. Monitoring jitter, which indicates variations in packet arrival times, is essential for detecting network anomalies. Unusually high jitter values often signify underlying issues or potential attacks within the network.

In our analysis, we consider various time periods, including 1, 5, 10, 30, and 60 s, for anomaly detection. This multitimeframe approach provides several advantages, such as improved anomaly detection, comprehensive behavior profiling, and greater resilience to noise. By evaluating features across different time scales, we capture both rapid, short-term anomalies and more subtle, long-term trends. However, this approach increases data size, which can be efficiently managed

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPLETE SET OF FLOW-BASED FEATURES FOR ANOMALY DETECTION IN IOT DEVICES USING CICFLOWMETER

| No | Feature name | No | Feature name | No | Feature name | No | Feature name |
|---|---|---|---|---|---|---|---|
| 1 | Flow ID | 22 | Flow Packets/s | 43 | Fwd Packets/s | 64 | Fwd Bulk Rate Avg |
| 2 | Src IP | 23 | Flow IAT Mean | 44 | Bwd Packets/s | 65 | Bwd Bytes/Bulk Avg |
| 3 | Src Port | 24 | Flow IAT Std | 45 | Packet Length Min | 66 | Bwd Packet/Bulk Avg |
| 4 | Dst IP | 25 | Flow IAT Max | 46 | Packet Length Max | 67 | Bwd Bulk Rate Avg |
| 5 | Dst Port | 26 | Flow IAT Min | 47 | Packet Length Mean | 68 | Subflow Fwd Packets |
| 6 | Protocol | 27 | Fwd IAT Total | 48 | Packet Length Std | 69 | Subflow Fwd Bytes |
| 7 | Timestamp | 28 | Fwd IAT Mean | 49 | Packet Length Variance | 70 | Subflow Bwd Packets |
| 8 | Flow Duration | 29 | Fwd IAT Std | 50 | FIN Flag Count | 71 | Subflow Bwd Bytes |
| 9 | Total Fwd Packet | 30 | Fwd IAT Max | 51 | SYN Flag Count | 72 | FWD Init Win Bytes |
| 10 | Total Bwd packets | 31 | Fwd IAT Min | 52 | RST Flag Count | 73 | Bwd Init Win Bytes |
| 11 | Total Length of Fwd Packet | 32 | Bwd IAT Total | 53 | PSH Flag Count | 74 | Fwd Act Data Pkts |
| 12 | Total Length of Bwd Packet | 33 | Bwd IAT Mean | 54 | ACK Flag Count | 75 | Fwd Seg Size Min |
| 13 | Fwd Packet Length Max | 34 | Bwd IAT Std | 55 | URG Flag Count | 76 | Active Mean |
| 14 | Fwd Packet Length Min | 35 | Bwd IAT Max | 56 | CWR Flag Count | 77 | Active Std |
| 15 | Fwd Packet Length Mean | 36 | Bwd IAT Min | 57 | ECE Flag Count | 78 | Active Max |
| 16 | Fwd Packet Length Std | 37 | Fwd PSH Flags | 58 | Down/Up Ratio | 79 | Active Min |
| 17 | Bwd Packet Length Max | 38 | Bwd PSH Flags | 59 | Average Packet Size | 80 | Idle Mean |
| 18 | Bwd Packet Length Min | 39 | Fwd URG Flags | 60 | Fwd Segment Size Avg | 81 | Idle Std |
| 19 | Bwd Packet Length Mean | 40 | Bwd URG Flags | 61 | Bwd Segment Size Avg | 82 | Idle Max |
| 20 | Bwd Packet Length Std | 41 | Fwd Header Length | 62 | Fwd Bytes/Bulk Avg | 83 | Idle Min |
| 21 | Flow Bytes/s | 42 | Bwd Header Length | 63 | Fwd Packet/Bulk Avg | 84 | Label |

through feature selection techniques before feeding the data into machine learning models for anomaly detection. The adaptability of features across different time periods allows for the detection of a wide range of anomalies. For instance, high counts in shorter intervals may indicate traffic surges, which are characterized by a sharp increase in network activity. Conversely, persistent threats, such as low-rate DoS attacks or slow data exfiltration attempts, are more effectively detected by observing gradual changes over extended periods. This flexibility in time-based analysis strengthens the reliability and accuracy of the anomaly detection process. Table II presents the average jitter values over different time intervals for various traffic types. For example, benign traffic starts with an average jitter value of 264.4 at the 1-s interval and gradually increases to 331.1 at the 60-s interval. Monitoring these changes across multiple time periods a deeper understanding of traffic behavior, allowing us to identify patterns that might be missed when analyzing data over a single, static timeframe. A visualization of these changes across multiple time periods is illustrated in Fig. 3. This layered approach to time-based analysis not only helps in identifying anomalies but also enhances the understanding of the dynamics of network traffic, which contributes to more effective anomaly detection strategies.

### *B. Feature Extraction Using Flow-Based Approach With CICFlowMeter*

In the anomaly detection phase, some features play an important roles in differentiating between normal and abnormal traffic. In order to extract effective features, we adopted two primary approaches. The first approach involves extracting features each packet individually, as explained in the previous section. The second approach considers each flow of packets as a unique object and extracts appropriate features from the entire flow. In this concept, a flow is defined as a sequence of packets that share common attributes, such as source IP, destination IP, and protocol, over a specified time period. Analyzing flows rather than individual packets allows us identify broader traffic patterns and anomalies that may occur over the duration of one communication session.

There are various methods for extracting features from a flow, and one particularly useful tool for this purpose, especially when dealing with PCAP files containing multiple flows, is CICFlowMeter [28]. CICFlowMeter is an open-source tool which is designed to extract over 80 network traffic statistical features from flow, including packet count, flow duration, and byte count, among others. These features are crucial for identifying anomalous behaviors in network traffic, particularly in complex environments, such as IoT networks, where traditional packet-level analysis might not be sufficient. In the following section, we explain this tool in more detail. By leveraging CICFlowMeter, we could enhance the our anomaly detection model's accuracy, which can be helpful in identifying and mitigating potential threats in IoT environments.

*Extracted Features Based on CICFlowmeter:* In order to extract some features, CICFlowMeter generates bidirectional flows, where the first packet establishes the forward direction (from source to destination) and the backward direction (from destination to source). This bidirectional flow analysis allows it to calculate statistical time-related features separately for both forward and backward directions which provides a comprehensive understanding of the flow [29]. In addition to these generic statistical features, CICFlowMeter also provides some advanced functionalities like the ability of selecting specific features from the existing list, adding specific features, and controlling the duration of flow timeouts. This flexibility makes CICFlowMeter a valuable tool for different network analysis scenarios. To provide a clearer understanding of the

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

features extracted from CICFlowmeter, we have categorized them into several groups, which are outlined as follows.

1) *Inter Arrival Time (IAT):* IAT is the first set of features that are crucial for network traffic analysis, since they are measured based on time intervals between sequential packets within a network flow. In this category, various statistical features, such as mean, minimum, maximum, and standard deviation, are extracted from the forward and backward IAT values. These features play an important role in identifying patterns withing a network traffic and detecting anomalies that may indicate potential security threats, such as DoS attacks or network scans. Some examples of features in this category include:
   a) *Flow IAT Mean:* This feature shows the average time between all packets in a flow. This feature is important since understanding the typical time delay between packets in a network session can indicate whether the network functions normally or is experiencing abnormality.
   b) *Flow IAT Std (Standard Deviation):* This feature measures time difference between two consecutive packets in a flow. A high IAT indicates significant fluctuations in packet timing, which may be a sign of unstable network conditions or inconsistent traffic behaviors.
   c) *Flow IAT Max (Maximum):* This feature shows the longest time interval between two packets in a flow. It can be a good sign for inconsistency in data transmission, such as delay.
   d) *Flow IAT Min (Minimum):* This feature measure the shortest time interval between two packets within a flow. Identifying the minimal delay between packets is an important factor in measuring best packet pace in a network.
2) *Flags:* In this feature category, features are extracted based on existence and frequency of specific TCP flags in packets. Each flag in a TCP connections has a unique function, and monitoring their occurrences can be helpful in identifying abnormalities like network attacks or misconfigurations. Some examples of features in this category include:
   a) *Fwd PSH Flags/Bwd PSH Flags:* These features measure how often the PSH flag appears in forward and backward packets, respectively. The PSH flag is used to specify that the data should be sent straight to the receiving application without buffering.
   b) *Fwd URG Flags/Bwd URG Flags:* These two features keep track of forward and backward URG (Urgent) flag. This flag notifies the recipient that some data in a packet should be prioritized over other data. It is commonly used in scenarios where immediate attention is necessary.
   c) *FIN Flag Count:* This feature measures the number of packets that have the FIN flag, which indicates the end of data transmission between two devices. By monitoring the frequency of FIN flags we can distinguish between the normal and abrupt terminations, which can be a sign of attacks.
   d) *SYN Flag Count:* SYN flag is essential for establishing TCP connections as part of the TCP handshake. Tracking SYN flag helps us to identify how many attempts happens and can be particularly useful in detecting SYN flood attacks or a form of DDoS attack where an attacker overwhelms a target system with a surge of SYN requests. Therefore, this feature can be an important factor in detecting these kinds of attacks
3) *Active/Idle:* To have a better understanding of the dynamics of network activity, Statistical features from both active and idle TCP flows play an important role, since it gives us more details about how long the system is active or idle and therefore, it can be helpful for detecting potential issues, such as botnet behavior or irregular traffic patterns in the system. Examples of features in this category as follows.
   a) *Active Min:* This feature records how long a network flow was active before changing to the idle state. Identifying the smallest active period can reveal frequent interruptions or high levels of network segmentation, which could be signs of aggressive session.
   b) *Active Mean:* This feature measures the average time that a flow remains active before switching to the idle mode. It can be helpful in identifying abnormal idle or active duration, such as unusually short or extended active periods.
   c) *Active Max:* This feature measures how long a flow remains active before switching to idle. Extended active periods can be considered normal for some types of data transfers, such as large file downloads or continuous data streams in applications like video streaming. This feature gives us good insights about the upper limits of flow activities and as a result detecting anomalies based on longer connections.
   d) *Active Std (Standard Deviation):* This feature calculates the variability in how long a flow remains active. A high value for this feature shows considerable differences between active durations of various flows, which may happen because of diverse group of applications.
4) *Segment, Bulk, Byte:* These set of features provide important insights about the size of data flows, which is important for understanding the efficiency of transmitted data over a network. These features are practical for evaluating how data are packaged and transmitted between endpoints and therefore for identifying potential bottlenecks or unusual data patterns.
   a) *Fwd Segment Size Avg:* This metric represents the average size of segments (groups of packets) that is transmitted in the forward direction. It helps in understanding the efficiency of the data payload in a network. Analyzing this feature plays an

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

important role in detecting anomalies in segment sizes.

b) *Bwd Segment Size Avg:* This feature indicates the average size of segments sent in the backward direction. Monitoring the backward segment size can also give us some information about behavior of the receiving end, which may be different based on the application, protocol used, or network conditions. Understanding these patterns is important in identifying unexpected behavior in data exchanges.

c) *Fwd Bytes/Bulk Avg:* This metric calculates the average number of bytes that is transferred per bulk in the forward direction. In this context, *Bulk* refers to a batch of packets that is sent together. Analyzing this feature is an important factor to assess the efficiency of data transmission, as it provides details about how much data are sent with each batch.

d) *Fwd Packet/Bulk Avg:* In this feature, the average number of packets per bulk in the forward direction is measured. It can be helpful in understanding how packets are grouped together during transmission, which reflects the transmission protocols or strategies employed. Lower values might suggest segmentation or buffering issues.

5) *Others:* CICFLowmeter provides other statistical features which can be extracted from packets and do not fall under the above categories. These features provide a variety of other aspects of packet behavior and transmission characteristics, which provides a comprehensive analysis of network traffic.

## IV. Experimental Results and Evaluation

This section presents the experimental findings obtained from analyzing Pcap files from the CICIoT2023 attack dataset. The analysis covers a diverse range of state-of-the-art attacks targeting IoT environments. Additionally, we discuss the rationale for conducting these experiments, the objectives we aimed to achieve, and the promising outcomes observed. The network attack classes included in the CICIoT2023 dataset comprise DoS, DDoS, Mirai, Reconnaissance, Brute Force, Spoofing, and Web-based attacks. These attacks represent some of the most prevalent threats that modern IoT environments must continuously defend against. Our experiments demonstrate how the generated features effectively detect these attacks, both individually and as a group.

Although this section primarily focuses on experiments related to anomaly detection in IoT devices, it also includes an additional experiment dedicated to device identification using the behavioral features generated from the same dataset. We demonstrate the potential to design an integrated, lightweight system of machine learning models that effectively addresses both anomaly detection and device identification challenges.

### A. Evaluation Metrics

In evaluating machine learning approaches, creating a confusion matrix is a common practice to assess classifier model effectiveness. This matrix summarizes both correctly and incorrectly classified patterns, incorporating essential metrics: true positive (TP), true negative (TN), FP, and false negative (FN). These metrics are particularly important for both binary classification in anomaly detection systems and multiclass classification in IoT device identification. In the following section, we will explain the state-of-the-art evaluation metrics used for both anomaly detection and device identification.

1) *Confusion Matrix:* A crucial tool for assessing the performance of both binary and multiclass classification systems. It enables the analysis of predictions in relation to actual classes, providing valuable insights into the classifier's accuracy across different categories.

2) *Overall Accuracy:* This metric evaluates the classifier's performance by calculating the percentage of correctly classified patterns (TPs and TNs) relative to the total number of patterns, including misclassified ones (FPs and FNs). The following equation illustrates the calculation of overall accuracy:

$$\text{Overall Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}. \quad (1)$$

3) *Recall:* This metric measures the proportion of true anomalies correctly identified by the model, which reflects its sensitivity in capturing TP cases. The following equation represents recall:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \quad (2)$$

4) *Precision:* This metric evaluates the accuracy of the model's positive predictions by measuring the proportion of true anomalies correctly identified among all instances predicted as anomalies. Precision is crucial for assessing the model's reliability in minimizing FPs. Precision is calculated using the following equation:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \quad (3)$$

5) $F_1$ *Score:* This metric provides a balanced evaluation of the model's performance by combining precision and recall, taking into account both FPs and FNs. For anomaly detection, the F1 score is particularly useful for balancing the identification of true anomalies and minimizing false alarms. The formula for the F1 score is given by

$$F_1 \text{ Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (4)$$

Additionally, in scenarios where two models yield similar scores across these metrics, the approach with lower training and testing times is preferred. This decision aligns with the objective of developing anomaly detection and classification models suitable for environments with limited computing resources. Low-latency prediction is crucial for ensuring efficiency in these environments. Therefore, in addition to traditional evaluation metrics, the computational efficiency of the models, particularly concerning the time required for training and inference, will be a key consideration in our model selection process.

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

### B. Evaluation Results

This section presents various classification experiments focused on anomaly detection and device identification in IoT environments. The first experiment, discussed in Section IV-B1, evaluates a standalone anomaly detection system that uses flow-based features to classify instances as either benign or attack. Section IV-B2 presents experimental results for an integrated model that classifies a wide range of IoT devices while simultaneously detecting anomalous behaviors. This integrated approach leverages behavior-based features systematically extracted from IoT network traffic which enable the system to both identify device types and distinguish between anomalous and benign device behaviors. To further assess the quality of the extracted features in distinguishing between anomalous and benign behaviors, specifically for previously unseen devices not included in the training phase, a scenario was designed to evaluate model performance against ZDAs. This evaluation, discussed in Section IV-B3, provides insights into the model's resilience when encountering new, unrecognized attack types. Additionally, throughout all experiments, the goal was to compare the performance of various machine learning classifier techniques to develop a lightweight model that achieves high accuracy with a minimal FP rate, while also ensuring rapid training and testing times. This approach is essential for deployment in IoT environments, where power and computational resources are often limited.

In addition to exploring lightweight machine learning classifiers, such as decision trees (DTs), random forest (RF), XGBoost, LR, and multilayer perceptron (MLP), various feature selection techniques were evaluated to optimize model performance. The classifiers were selected for their efficiency and compatibility with resource-constrained IoT environments. Hyperparameters, including the number of trees in RF and the learning rate in XGBoost, were tuned to maximize performance. For feature selection, ANOVA was used to assess the statistical significance of features, while the Pearson correlation coefficient (PCC) helped identify and remove highly correlated features. Additionally, Genetic Algorithms and recursive feature elimination (RFE) were applied to iteratively select an optimal subset of features that balance model accuracy and computational efficiency. In most experiments, the XGBoost classifier combined with RFE demonstrated superior performance compared to other machine learning techniques and feature selection methods. This analysis provides insights into the tradeoffs between evaluation metric scores and training/testing times, ensuring that the approaches remain efficient.

As mentioned earlier, we utilized the network traffic data (PCAP files) from the state-of-the-art CICIoT2023 dataset. This dataset provides real samples from a variety of IoT devices and offers a proportional representation of both benign instances and all attack classes. All experiments were conducted using a 70% train-test split. In the ZDA experiments, one specific attack class was excluded from the training data, ensuring that it was only present in the testing dataset alongside benign instances. The preprocessing steps implemented in this study were uniform across all experiments. To handle null values in the dataset, we implemented a strategy that involved separately addressing categorical and numerical columns. For numerical columns, we applied scaling using *MinMaxScaler* and replaced null values with the mean of the column. In case of categorical columns, we imputed null values by substituting them with the most frequent value in the column. These methods were applied to each category of attack, specifically in the benign data samples. This approach ensures that the more prevalent attack classes do not influence the handling of null values in the less represented classes in our dataset.

In addition to the anomaly detection approaches mentioned above, we also explored the effectiveness of the behavioral features generated from the CICIoT2023 dataset for device classification. This experiment aimed to identify the various devices present in the dataset using behavioral features. Our findings indicated that a DT-based classifier outperformed the other machine learning classifiers utilized in the study. We also employed RFE to obtain a reduced subset of 70 features (out of 133 features listed in Table III), using a DT classifier to evaluate their performance. Similar to the anomaly detection results, we observed only a minor reduction in evaluation metrics, accompanied by a decrease in training and testing times.

In conclusion, we developed a hybrid system, demonstrating that a cost-effective integrated model leveraging behavioral features is both feasible and effective. The device identification task is performed using a DT-based classifier, while the anomaly detection component employs an XGBoost-based classifier, ensuring both robust performance and reliability. In the CICIoT2023 attack dataset, we identified 40 different device classes with sufficient representation, allowing them to be effectively utilized in the experiments. All experiments were conducted on a machine equipped with a 13th Gen Intel Core i7-1360P processor (2.20 GHz base clock), 16 GB of DDR4 RAM (15.7 GB usable), running Windows 11 Enterprise, Version 22H2 (OS build 22621.4037), and Windows Feature Experience Pack 1000.22700.1027.0. The system is a 64-bit operating system on an x64-based processor. The algorithms and data processing steps were executed using Python, with the key libraries being *NumPy*, *scikit-learn*, and *pandas*.

*1) Performance Results of Standalone Anomaly Detection Model Utilizing Flow-Based Analysis:* In this experiment, supervised learning classifiers were applied to flow-based features, using a 70:30 train-test split on a dataset with over 2.3 million samples, encompassing both benign device samples and samples from various attack categories as detailed in Section I-A. Table IV lists 84 flow-based features for both individual attack detection and anomaly detection, with 81 features (excluding Flow ID and Timestamp) selected for evaluation. The results of this evaluation are presented in Table V. This table illustrates the performance results of individual attack detection against benign samples using the XGBoost classifier. Additionally, the bottom rows of the table (highlighted in blue) show the performance of various supervised learning classifiers for anomaly detection, which is a balanced combination of samples from all attack categories against benign samples.

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

TABLE V
EXPERIMENTAL RESULTS FOR ANOMALY DETECTION AND ATTACK CLASSIFICATION USING FLOW-BASED FEATURES

| Attack Type | Classifier Technique | Train Time (s) | Test Time (s) | No of samples | No of features | FS Technique | Accuracy | Precision (MA) | Recall (MA) | $F_1$ Score (MA) |
|---|---|---|---|---|---|---|---|---|---|---|
| **Brute Force** | XGBoost | 7.47 | 1.51 | 401,949 | 81 (all) | - | 0.9961 | 0.9904 | 0.7932 | 0.8666 |
| | XGBoost | 4.45 | 0.50 | 401,949 | 40 | **RFE** | 0.9960 | 0.9908 | 0.7840 | 0.8593 |
| **DDoS** | XGBoost | 10.46 | 2.37 | 1,094,093 | 81 (all) | - | 0.9696 | 0.9653 | 0.9694 | 0.9673 |
| | XGBoost | 8.67 | 2.25 | 1,094,093 | 40 | **RFE** | 0.9691 | 0.9648 | 0.9689 | 0.9668 |
| **DoS** | XGBoost | 9.90 | 3.94 | 861,208 | 81 (all) | - | 0.9984 | 0.9985 | 0.9984 | 0.9984 |
| | XGBoost | 7.35 | 2.04 | 861,208 | **40** | **RFE** | 0.9985 | 0.9985 | 0.9985 | 0.9985 |
| **Mirai** | XGBoost | 5.93 | 1.78 | 572,918 | 81 (all) | - | 0.9988 | 0.9987 | 0.9986 | 0.9985 |
| | XGBoost | 4.36 | 0.81 | 572,918 | **40** | **RFE** | 0.9987 | 0.9986 | 0.9983 | 0.9985 |
| **Spoofing** | XGBoost | 5.31 | 1.09 | 555,568 | 81 (all) | - | 0.9964 | 0.9961 | 0.9949 | 0.9955 |
| | XGBoost | 4.21 | 0.82 | 555,568 | **40** | **RFE** | 0.9964 | 0.9962 | 0.9950 | 0.9956 |
| **Web-Based** | XGBoost | 12.24 | 2.30 | 851,816 | 81 (all) | - | 0.9260 | 0.9255 | 0.9260 | 0.9257 |
| | XGBoost | 6.35 | 1.07 | 851,816 | **40** | **RFE** | 0.9257 | 0.9252 | 0.9258 | 0.9254 |
| **All attacks** | XGBoost | 31.07 | 5.84 | 2,345,902 | 81 (all) | - | 0.9592 | 0.9260 | 0.9302 | 0.9281 |
| | XGBoost | 19.52 | 2.99 | 2,345,902 | **40** | **RFE** | 0.9585 | 0.9251 | 0.9285 | 0.9268 |
| **All attacks** | Random Forest | 109.68 | 6.41 | 2,345,902 | 81 (all) | - | 0.9590 | 0.9276 | 0.9269 | 0.9272 |
| | Random Forest | 107.21 | 5.76 | 2,345,902 | 40 | **RFE** | 0.9579 | 0.9260 | 0.92643 | 0.9252 |
| **All attacks** | Decision Tree | 59.68 | 3.61 | 2,345,902 | 81 (all) | - | 0.9502 | 0.9148 | 0.9072 | 0.9109 |
| | Decision Tree | 37.93 | 3.02 | 2,345,902 | **40** | **RFE** | 0.9500 | 0.9144 | 0.9907 | 0.9106 |
| **All attacks** | Multi-Layer Perceptron | 1174.77 | 17.06 | 2,345,902 | 81 (all) | - | 0.9356 | 0.8940 | 0.8724 | 0.8827 |
| | Multi-Layer Perceptron | 277.23 | 11.45 | 2,345,902 | **40** | **RFE** | 0.9342 | 0.9074 | 0.8490 | 0.8745 |
| **All attacks** | Logistic Regression | 70.66 | 4.45 | 2,345,902 | 81 (all) | - | 0.9229 | 0.8734 | 0.8441 | 0.8578 |
| | Logistic Regression | 30.16 | 2.87 | 2,345,902 | **40** | **RFE** | 0.9222 | 0.8720 | 0.8431 | 0.8566 |

**Note:** **MA=** Macro Average, **RFE=** Recursive Feature Elimination.

As observed in this table, XGBoost proved to be the best performing classifier among all the supervised learning classifiers, achieving the highest scores across evaluation metrics and demonstrating the fastest training time with minimal testing duration. Out of a total of 2 345 902 samples in the dataset, the XGBoost technique was trained in 19.5 s on 70% of the data and took 2.99 s for testing on the remaining data, based on 40 features selected using the RFE technique. This model achieved an accuracy of 0.9585, precision of 0.9251, recall of 0.9285, and an F1 score of 0.9268, reflecting its robust performance.

For the experiments focused on individual attack detection against benign samples, the training and testing subsets exclusively contained instances from those specific classes. Datasets with sufficiently large samples from attack classes yielded strong performance; however, in the case of the Brute Force attack experiments, the limited number of available samples negatively impacted the F1 scores. For Web-Based attacks, all metric scores were around 92%. The experiment concentrating on Mirai attacks demonstrated that the XGBoost classifier achieved an exceptional accuracy of 99.88% which highlights the effectiveness of flow-based features in detecting this well-known IoT-targeted malware. Additionally, by employing RFE with XGBoost, we identified the top 40 features to enhance model efficiency and interpretability, as previously described. The performance of models trained with this reduced feature subset showed evaluation metric scores very close to those of models trained with the full set of features, while also significantly reducing training and testing times.

*2) Performance Results for Anomaly Detection and Attack Classification Using Packet-Based Approach:* In the previous section, the extracted features were suitable only for the task of anomaly detection and were not capable of identifying device categories. However, for an efficient access management system in IoT environments, it is crucial to analyze and detect anomalous behaviors alongside recognizing device categories. Furthermore, for specific attack categories with a limited number of training samples, such as Brute Force and Web-Based attacks, the performance results (particularly the F1 score) were unsatisfactory, which could negatively impact the effectiveness of the detection system. To address this issue, we extracted 133 behavior-based features using a different packet-by-packet approach (discussed in Section III), which enable us to effectively accomplish both IoT device classification and anomaly detection tasks. The results of this experiment are presented in Table VI. The number of available samples and the set of features are larger compared to the flow-based features, which ultimately results in models with improved scores in evaluation metrics compared to previous experiments.

The individual attack detection models for each attack category in the CICIoT2023 dataset, as shown in the top rows of Table VI, exhibited enhanced performance across all evaluation metrics, with notably faster training and testing times. These results highlight that the quality of the extracted features provides a strong and accurate representation of the attack behaviors observed on the device side. For individual attack detection, 900 000 benign samples and 300 000 samples per attack category from the dataset were used, except for the Brute Force attack category, which had fewer samples (231 477) available. This distribution reflects real-world network traffic, where benign samples are considerably more frequent than attack samples to improve the model's ability to detect malicious behavior within typical network activity.

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
EXPERIMENTAL RESULTS FOR ATTACK CLASSIFICATION AND ANOMALY DETECTION USING PACKET-BASED FEATURES

| Attack Type | Classifier Technique | Train Time (s) | Test Time (s) | No of samples | No of features | FS Technique | Accuracy | Precision (MA) | Recall (MA) | $F_1$ Score (MA) |
|---|---|---|---|---|---|---|---|---|---|---|
| **Brute Force** | XGBoost | 42.88 | 3.57 | 1,031,477 | 133 (all) | - | 0.9999 | 0.9999 | 0.9997 | 0.9998 |
| | XGBoost | 28.54 | 2.00 | 1,031,477 | **70** | **RFE** | 0.9998 | 0.9999 | 0.9997 | 0.9999 |
| **DDoS** | XGBoost | 41.01 | 3.42 | 1,200,000 | 133 (all) | - | 0.9999 | 0.9999 | 0.9999 | 0.9999 |
| | XGBoost | 30.64 | 1.85 | 1,200,000 | **70** | **RFE** | 0.9999 | 0.9999 | 0.9999 | 0.9999 |
| **DoS** | XGBoost | 44.81 | 3.19 | 1,200,000 | 133 (all) | - | 0.9999 | 0.9999 | 0.9999 | 0.9999 |
| | XGBoost | 19.06 | 1.98 | 1,200,000 | **70** | **RFE** | 0.9999 | 0.9999 | 0.9999 | 0.9999 |
| **Reconnaissance** | XGBoost | 44.77 | 2.97 | 1,200,000 | 133 (all) | - | 0.9999 | 0.9998 | 0.9998 | 0.9998 |
| | XGBoost | 19.09 | 1.97 | 1,200,000 | **70** | **RFE** | 0.9997 | 0.9998 | 0.9997 | 0.9998 |
| **Mirai** | XGBoost | 41.89 | 2.98 | 1,200,000 | 133 (all) | - | 1.0 | 1.0 | 1.0 | 1.0 |
| | XGBoost | 35.15 | 1.88 | 1,200,000 | **70** | **RFE** | 1.0 | 1.0 | 1.0 | 1.0 |
| **Spoofing** | XGBoost | 46.04 | 3.54 | 1,200,000 | 133 (all) | - | 0.9999 | 0.9999 | 0.9999 | 0.9999 |
| | XGBoost | 32.82 | 1.76 | 1,200,000 | **70** | **RFE** | 0.9999 | 0.9999 | 0.9999 | 0.9999 |
| **Web-Based** | XGBoost | 42.09 | 3.27 | 1,153,902 | 133 (all) | - | 0.9997 | 0.9997 | 0.9995 | 0.9996 |
| | XGBoost | 35.50 | 3.48 | 1,153,902 | **70** | **RFE** | 0.9995 | 0.9996 | 0.9995 | 0.9997 |
| **All attacks** | XGBoost | 75.85 | 9.80 | 2,785,379 | 133 (all) | - | 0.9995 | 0.9994 | 0.9995 | 0.9994 |
| | XGBoost | 25.01 | 4.41 | 2,785,379 | **70** | **RFE** | 0.9994 | 0.9993 | 0.9994 | 0.9994 |
| **All attacks** | Decision Tree | 168.78 | 9.20 | 2,785,379 | 133 (all) | - | 0.9989 | 0.9987 | 0.9988 | 0.9987 |
| | Decision Tree | 91.68 | 3.83 | 2,785,379 | **70** | **RFE** | 0.9990 | 0.9988 | 0.9988 | 0.9988 |
| **All attacks** | Random Forest | 199.11 | 17.27 | 2,785,379 | 133 (all) | - | 0.9995 | 0.9994 | 0.9995 | 0.9994 |
| | Random Forest | 154.20 | 6.35 | 2,785,379 | **70** | **RFE** | 0.9996 | 0.9995 | 0.9995 | 0.9995 |
| **All attacks** | Mutli-Layer Perceptron | 2133.49 | 22.22 | 2,785,379 | 133 (all) | - | 0.9943 | 0.9935 | 0.9935 | 0.9935 |
| | Mutli-Layer Perceptron | 582.36 | 14.62 | 2,785,379 | **70** | **RFE** | 0.9935 | 0.9915 | 0.9938 | 0.9926 |
| **All attacks** | Logistic Regression | 196.66 | 14.74 | 2,785,379 | 133 (all) | - | 0.9565 | 0.9466 | 0.9553 | 0.9507 |
| | Logistic Regression | 99.96 | 4.35 | 2,785,379 | **70** | **RFE** | 0.9543 | 0.9437 | 0.9534 | 0.9483 |

**Note: MA=** Macro Average, **RFE=** Recursive Feature Elimination.

In this experiment, various feature selection techniques and machine learning classifiers were applied, finding that RFE with 70 features and XGBoost yielded superior performance with high evaluation metrics across all cases. Notably, for the Mirai attack, a well-known IoT-targeted malware, the XGBoost model successfully detected 100% of the attack samples in the testing dataset without any misclassification errors. For DDoS attacks, the classifier achieved nearly perfect results, with accuracy, precision, recall, and F1 score all at 99.99%. Similarly, DoS attacks yielded identical high metrics, which emphasize the model's effectiveness in identifying both large-scale distributed attacks and localized denial-of-service events. These results are particularly significant given the complexity and variability of different attack behaviors, which further highlight the effectiveness of behavioral feature analysis in identifying sophisticated attacks. In contrast, Web-Based attacks were slightly more challenging to detect, with the XGBoost classifier achieving a still impressive F1 score of 99.96%. This slight reduction in performance compared to other attacks suggests that while behavioral features are highly effective, the complexity and subtlety of Web-based threats require even more refined detection techniques.

For anomaly detection, we created a balanced subset of samples by combining attack samples from all available categories in the CICIoT2023 dataset. The results for this experiment are presented in the bottom rows of Table VI, highlighted in blue. In this experiment, we evaluated the performance of various machine learning classifiers to facilitate a comparison and identify the best model for the anomaly detection phase. We utilized a substantial dataset containing 2 785 379 samples and presented results for both the complete set of 133 features and the 70 features selected using RFE. As indicated in this table, the XGBoost classifier, utilizing features selected based on RFE, outperformed other machine learning classifiers, including DT, RF, MLP, and LR, in the anomaly detection task. It achieved a higher F1 score along with significantly faster training and testing times. Specifically, it took only 25 s to train on 70% of the 2 785 379 samples and just 4.4 s for the remaining 30% during the testing phase.

*3) Evaluating Resilience Against Zero-Day Attacks in Anomaly Detection:* The purpose of the ZDA experiments is to evaluate how a model trained on various attack classes performs against an attack class that was not included in the training dataset. We conducted a series of seven experiments in which one attack class was excluded from the model during training. The isolated class, along with benign samples, was used to create the testing dataset. The experimental pipeline is illustrated in Fig. 6. The model is designed to evaluate how well it can differentiate benign traffic from all attack classes, even when those classes were not included in the training phase. Similar to the previous experiments, these experiments were conducted using the XGBoost classifier, which demonstrated the best performance in prior anomaly detection assessments. The results are summarized in Table VII. The selection of the XGBoost classifier for these experiments was based on its proven effectiveness in earlier evaluations. The findings from these experiments were particularly insightful. Additionally, the subset of features identified in the previous section using RFE with XGBoost was employed to evaluate the efficacy of those features in the context of ZDAs.

As illustrated in Fig. 6, Brute Force samples were entirely excluded from the training data, so the XGBoost model was

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

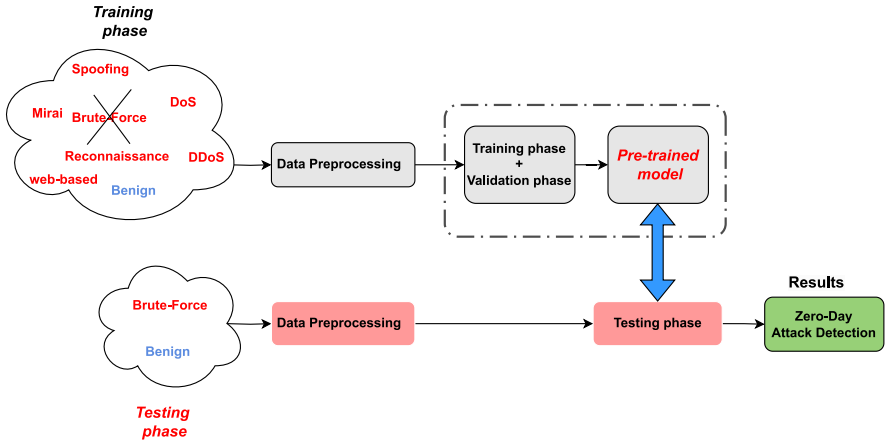



Fig. 6. Scenario for ZDA detection.

TABLE VII
EXPERIMENTAL RESULTS FOR ZDA DETECTION USING THE SELECTED CLASSIFIER AND PACKET-BASED FEATURES

| Zero-day Attack (Used for Testing) | Classifier Technique | Train Time (s) | Test Time (s) | No of samples | No of features | FS technique | Accuracy | Precision (MA) | Recall (MA) | $F_1$ Score (MA) |
|---|---|---|---|---|---|---|---|---|---|---|
| **Brute Force** | XGBoost | 101.88 | 8.43 | 2,785,379 | 133 (all) | - | 0.9964 | 0.9970 | 0.9946 | 0.9958 |
| | XGBoost | 54.68 | 5.97 | 2,785,379 | **70** | **RFE** | 0.9937 | 0.9950 | 0.9902 | 0.9926 |
| **DDoS** | XGBoost | 112.87 | 7.75 | 2,785,379 | 133 (all) | - | 0.9993 | 0.9993 | 0.9993 | 0.9993 |
| | XGBoost | 54.83 | 16.85 | 2,785,379 | **70** | **RFE** | 0.9993 | 0.9993 | 0.9993 | 0.9993 |
| **DoS** | XGBoost | 93.18 | 9.00 | 2,785,379 | 133 (all) | - | 0.9928 | 0.9928 | 0.9928 | 0.9928 |
| | XGBoost | 36.62 | 3.82 | 2,785,379 | **70** | **RFE** | 0.9885 | 0.9885 | 0.9885 | 0.9885 |
| **Reconnaissance** | XGBoost | 102.06 | 9.58 | 2,785,379 | 133 (all) | - | 0.9296 | 0.9382 | 0.9296 | 0.9293 |
| | XGBoost | 35.65 | 4.14 | 2,785,379 | **70** | **RFE** | 0.9306 | 0.9389 | 0.9306 | 0.9303 |
| **Mirai** | XGBoost | 116.83 | 8.54 | 2,785,379 | 133 (all) | - | 0.9996 | 0.9996 | 0.9996 | 0.9996 |
| | XGBoost | 38.74 | 3.35 | 2,785,379 | **70** | **RFE** | 0.9994 | 0.9994 | 0.9994 | 0.9994 |
| **Spoofing** | XGBoost | 99.79 | 8.10 | 2,785,379 | 133 (all) | - | 0.9755 | 0.9766 | 0.9755 | 0.9755 |
| | XGBoost | 43.61 | 3.50 | 2,785,379 | **70** | **RFE** | 0.9672 | 0.9691 | 0.9672 | 0.9672 |
| **Web-Based** | XGBoost | 98.15 | 7.96 | 2,785,379 | 133 (all) | - | 0.9323 | 0.9444 | 0.9262 | 0.9308 |
| | XGBoost | 49.76 | 4.82 | 2,785,379 | **70** | **RFE** | 0.9377 | 0.9483 | 0.9321 | 0.9364 |

**Note:** **MA=** Macro Average, **RFE=** Recursive Feature Elimination.

initially trained only on DDoS, DoS, Reconnaissance, Mirai, Spoofing, and Web-Based attacks. The model achieved an accuracy of 99.64% and F1 scores 99.58%, which demonstrate a strong capacity to detect brute force attempts even without prior exposure to this specific attack type during training. Even with the RFE-selected subset of 70 features, performance remained robust, with only a slight decrease in metrics. This outcome underscores the effectiveness of feature reduction in maintaining model efficiency without significantly compromising detection capabilities.

Against Mirai and DDoS attacks, the classifier excelled in detecting zero-day DDoS attacks, achieving near-perfect scores of 99.96% and 99.93%, respectively, across all metrics, both with the full set of 133 features and the reduced 70-feature set. These results indicate the model's ability to generalize the behavioral patterns associated with these attacks, even when encountering previously unseen variations. Similarly, the model performed strongly for DoS and Spoofing attacks, achieving F1 scores of 99.28% and 97.55%, respectively. However, a slight decrease in F1 scores for both DoS and Spoofing attacks was observed when using the reduced feature set, which highlights the potential challenges in maintaining detection accuracy with fewer features for these types of attacks.

Regarding reconnaissance and Web-Based attacks, the model faced more challenges, achieving F1 scores of 92.93% and 93.08%, respectively. This outcome suggests that while the model is generally effective for various attack patterns, reconnaissance and Web-Based activities present more sophisticated challenges for zero-day detection. It indicates that having at least a portion of these attack patterns included in the training phase may be necessary to enhance the model's performance in identifying such threats.

*4) Performance Results for IoT Device Classification:* In the device identification phase, as illustrated in Fig. 1, we extracted a distinct set of features exclusively for classifying different types of IoT device categories present in the CICIoT2023 attack dataset. Specifically, we identified a behavioral set of features, listed in Table III (feature 1–57), to provide a behavioral representation for each device category. The device identification problem employs a multiclass classification strategy using the *device_mac* behavioral feature as the target variable. In the CICIoT2023 dataset, we identified 45 different devices, each with at least 10 000 samples (up to

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
EXPERIMENTAL RESULTS FOR IOT DEVICE IDENTIFICATION USING PACKET-BASED FEATURES

| Classifier Technique | Train Time (s) | Test Time (s) | No of samples | No of device classes | No of features | FS technique | Accuracy | Precision (MA) | Recall (MA) | $F_1$ Score (MA) |
|---|---|---|---|---|---|---|---|---|---|---|
| **XGBoost** | 956.15 | 10.71 | 2,723,778 | 45 | 133 (all) | - | 0.9997 | 0.9998 | 0.9997 | 0.9997 |
| | 472.50 | 3.88 | 2,723,778 | 45 | **70** | **RFE** | 1.0 | 1.0 | 1.0 | 1.0 |
| **Random Forest** | 208.41 | 15.55 | 2,723,778 | 45 | 133 (all) | - | 0.9990 | 0.9991 | 0.9988 | 0.9990 |
| | 114.32 | 14.73 | 2,723,778 | 45 | **70** | **RFE** | 0.9997 | 0.9997 | 0.9996 | 0.9997 |
| **Decision Tree** | 195.15 | 6.32 | 2,723,778 | 45 | 133 (all) | - | 0.9993 | 0.9993 | 0.9992 | 0.9993 |
| | 56.08 | 2.05 | 2,723,778 | 45 | **70** | **RFE** | 0.9999 | 0.9999 | 0.9999 | 0.9999 |
| **MultiLayer Perceptron** | 3738.99 | 12.46 | 2,723,778 | 45 | 133 (all) | - | 0.9844 | 0.9635 | 0.9766 | 0.9665 |
| | 1111.83 | 6.12 | 2,723,778 | 45 | **70** | **RFE** | 0.9998 | 0.9996 | 0.9995 | 0.9995 |
| **Logistic Regression** | 1440.69 | 5.27 | 2,723,778 | 45 | 133 (all) | - | 0.8805 | 0.7739 | 0.7348 | 0.7473 |
| | 714.32 | 2.12 | 2,723,778 | 45 | **70** | **RFE** | 0.9524 | 0.9257 | 0.8828 | 0.8990 |

**Note: MA=** Macro Average, **RFE=** Recursive Feature Elimination.

TABLE IX
EXPERIMENTAL RESULTS FOR THE INTEGRATED MODEL FOR BOTH DEVICE IDENTIFICATION AND ANOMALY DETECTION USING PACKET-BASED FEATURES

| Task | Classifier Technique | Train Time (s) | Test Time (s) | No of samples | No of features | FS Technique | No of classes | Accuracy | Precision (MA) | Recall (MA) | $F_1$ Score (MA) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Device Identification** | Decision Tree | 195.15 | 6.32 | 2,723,778 | 133 (all) | - | 45 | 0.9993 | 0.9993 | 0.9992 | 0.9993 |
| | Decision Tree | 56.08 | 2.05 | 2,723,778 | **37** | **RFE** | 45 | 0.9999 | 0.9999 | 0.9999 | 0.9999 |
| **Anomaly Detection** | XGBoost | 75.85 | 9.80 | 2,785,379 | 133 (all) | - | 2 | 0.9994 | 0.9993 | 0.9994 | 0.9994 |
| | XGBoost | 18.43 | 1.41 | 2,723,778 | **37** | **RFE** | 2 | 0.9994 | 0.9989 | 0.9991 | 0.9990 |

**Note: MA=** Macro Average, **RFE=** Recursive Feature Elimination.

100 000 samples were selected for the more prevalent classes). This selection resulted in a dataset comprising 2.72 million samples. Similar to the anomaly detection phase, we conducted trials using both the complete set of 133 features and a reduced set of features selected through RFE. Both sets of experiments were performed using a 70:30 train-test split.

Similar to the previous experiments for anomaly detection, we employed various lightweight supervised learning techniques for device identification, utilizing the complete set of behavioral features to assess their efficacy in distinguishing the 45 different classes. Additionally, we conducted experiments to identify the fastest and most accurate classifier using the 70 features selected by RFE, which allows us to perform a comparative analysis of performance between the full feature set and the reduced feature set.

Through multiple experiments, we found that the DT classifier outperformed XGBoost in terms of training and testing speed, while achieving nearly same performance on evaluation metrics. Although XGBoost achieved perfect results for device identification when trained on all features, its training and testing times were significantly higher, at 956 s for training and 10.7 s for testing, compared to the DT's 195 s for training and 6.3 s for testing. Interestingly, when using the reduced 70 features, XGBoost maintained perfect performance across all metrics, with 100% accuracy, precision, recall, and F1 score, without any misclassifications.

This result highlights XGBoost's robustness, as it maintains high performance even with a reduced feature set, making it an ideal choice for real-time IoT applications where accuracy is paramount. However, when considering both accuracy and time complexity, the DT classifier trained with 70 features emerges as an excellent model. It provides significantly lower training and testing times (56.08 s for training and 2.05 s for testing) compared to XGBoost, while delivering nearly identical evaluation results, achieving a score of 0.9999 across all metrics. This makes the DT model particularly well-suited for environments where both speed and accuracy are critical, such as in edge computing scenarios. The results for device identification are summarized in Table VIII.

*5) Performance Results for the Integrated Model for Detecting Anomalies and Classifying IoT Devices Based on Behavioral Analysis:* This section describes the integrated models for simultaneous device identification and anomaly detection. In this framework, an access management system processes a comprehensive behavioral feature set, with two classifiers running in parallel. These models concurrently identify devices and detect anomalies based on real-time traffic data, and provide an efficient solution for enhanced access control and security. As discussed earlier, DT-based classifier was identified as the top-performing model for device identification, while XGBoost-based classifiers demonstrated superior performance for anomaly detection. These two classifiers were trained using the behavioral features listed in Table III, and the results for both tasks are summarized in Table IX.

In summary, the trained DT model demonstrated exceptional efficiency, accurately classifying 45 device categories with evaluation metrics of 0.9999 across all measures, including accuracy, precision, recall, and F1 score. Additionally, for anomaly detection, the XGBoost classifier effectively distinguished between attacks and benign traffic, achieving high efficiency with scores of 0.999 across all evaluation metrics. Together, these findings underscore the effectiveness of the integrated approach, where the dual-classification strategy enhances overall system performance. This comprehensive

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

framework provides reliable device identification and anomaly detection, which is crucial for maintaining security and efficiency in modern IoT environments.

To further enhance overall efficiency and reduce complexity, we employed a subset of 37 features common to both RFE-generated feature sets for the two classification tasks. By concentrating on this reduced feature subset, we trained models that demonstrated significantly shorter training and testing times while maintaining high-quality results. These models optimize resource utilization, making them particularly advantageous for deployment in environments with limited computational power, such as edge computing scenarios. This approach enables the development of a lightweight model capable of operating efficiently while still delivering robust classification performance across all evaluation metrics.

### C. Discussion and Comparison

The experimental results presented in the previous section, evaluated using real network traffic data from IoT devices collected in the IoT laboratory of the Canadian Institute for Cybersecurity, demonstrate that the proposed hybrid system outperforms existing methods in terms of accuracy and computational efficiency. The proposed integrated machine learning model can be deployed directly on the home router, which acts as the central hub for all network traffic in a typical smart home environment. Since IoT devices usually do not connect directly to the wide area network (WAN), all their inbound and outbound traffic passes through the home router. This configuration enables the router to capture and analyze all IoT traffic without additional network devices or complex configurations. To address the challenge of inspecting encrypted traffic, features were collected specifically from encrypted communications, with a focus on HTTPS-related attributes, such as handshake information and user agent strings. These features enable effective monitoring and analysis of encrypted traffic without requiring full packet-level inspection. The proposed approach aligns with the topology used in the CICIoT2023 dataset, which simulates an LAN environment using multiple network devices, including routers and switches, to replicate a real-world smart home network. By deploying ML models on the home router, we leverage its central position to monitor all network traffic, similar to how the CICIoT2023 topology utilizes a network tap and routers for traffic monitoring.

To assess the effectiveness of the proposed hybrid system, we compare it with existing state-of-the-art approaches in IoT device identification and anomaly detection. Although we could not find a hybrid approach that simultaneously classifies various IoT devices, detects anomalies, and classifies attack types, we made comparisons with individual models designed specifically for either device identification or anomaly detection. The results of this comparison are presented in Table X. For anomaly detection, the results were compared with recent studies that utilized the CICIoT2023 dataset, which was also used in this work. However, for IoT device identification, since the CICIoT2023 dataset primarily focuses on attack scenarios rather than device identification. Instead, comparisons were made with recent studies that used the CICIoT2022 device profiling dataset, which was collected in the same environment at the Canadian Institute for Cybersecurity.

TABLE X
COMPARATIVE RESULTS AGAINST RECENT ANOMALY DETECTION AND DEVICE IDENTIFICATION TECHNIQUES

| Reference | DI | AD | Accuracy | Recall | Precision | $F_1$ score | Dataset |
|---|---|---|---|---|---|---|---|
| Ruijie et al. [30] | Yes | No | 0.985 | 0.983 | 0.985 | 0.984 | CICIoT2022 |
| Bakhsh et al. [31] | Yes | No | 0.9642 | 0.9642 | 0.9642 | 0.9642 | CICIoT2022 |
| Zhao et al. [32] | Yes | No | 0.9874 | 0.9722 | 0.9868 | 0.9784 | CICIoT2022 |
| Behera et al. [33] | No | Yes | 0.9978 | 0.9876 | 100 | 0.9845 | CICIoT2023 |
| Jeffrey et al. [34] | No | Yes | 0.9319 | 0.9319 | 0.9353 | 0.9324 | CICIoT2023 |
| Zohourian et al. [35] | No | Yes | 0.9874 | 0.9971 | 0.9384 | 0.9529 | CICIoT2023 |
| **Proposed DI Approach** | **Yes** | **-** | **0.9999** | **0.9999** | **0.9999** | **0.9999** | CICIoT2023 |
| **Proposed AD Approach** | **-** | **Yes** | **0.9994** | **0.9991** | **0.9989** | **0.9990** | CICIoT2023 |

Note: **DI:** Device Identification, **AD:** Anomaly Detection.

## V. CONCLUSION

This article presents a lightweight framework for integrated device identification and anomaly detection in IoT environments, focusing primarily on extracting behavioral-based features that reduce computational demands. The proposed approach utilizes lightweight features that accurately capture device behaviors with minimal computational overhead, to make it particularly suitable for deployment in resource-constrained IoT environments. Extracting relevant and significant features is crucial for constructing an efficient and effective anomaly detection system. To achieve this, we explored two complementary methods: 1) a packet-based approach and 2) a flow-based approach to analyze network traffic data. These methodologies enable us to obtain a diverse set of features essential for ensuring robust anomaly detection and device identification. After extracting significant features capable of distinguishing attack samples from benign ones in IoT devices, we employed a diverse set of lightweight machine learning models for training and comparison. The goal was to identify the fastest and most accurate classifier for both anomaly detection and device identification tasks. This dual-functionality model improves both efficiency and security, addressing the need for effective, real-time IoT threat detection and device management. For future research, the adaptability of the proposed method to the rapidly evolving IoT landscape will be a key area of focus, particularly in addressing the increasing variety and complexity of devices. The scalability and applicability of the framework in more complex network environments will also be explored to assess its broader deployment potential. Additionally, further work will also focus on implementing explainability techniques to interpret the classifier's decisions, emphasizing key features that influence the model's predictions and ensuring greater transparency and trust in the system.

## DATA AVAILABILITY

The generated datasets will be shared in the page of the institution.

## ACKNOWLEDGMENT

The authors express their gratitude to the anonymous reviewers for their valuable feedback.

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.

## REFERENCES


[1] S. Elhadi, A. Marzak, N. Sael, and S. Merzouk, "Comparative study of IoT protocols," in *Proc. 2nd Int. Conf. Smart Appl. Data Anal. Smart Cities (SADASC)*, 2018, pp. 1–5.

[2] P. McFedries, *Alexa for Dummies*. Hoboken, NJ, USA: Wiley, 2021.

[3] R. Chataut, A. Phoummalayvane, and R. Akl, "Unleashing the power of IoT: A comprehensive review of IoT applications and future prospects in healthcare, agriculture, smart homes, smart cities, and industry 4.0," *Sensors*, vol. 23, no. 16, p. 7194, 2023.

[4] J. Mohanty, S. Mishra, S. Patra, B. Pati, and C. R. Panigrahi, "IoT security, challenges, and solutions: A review," in *Proc. Prog. Adv. Comput. Intell. Eng. (ICACIE)*, 2021, pp. 493–504.

[5] M. Noura, M. Atiquzzaman, and M. Gaedke, "Interoperability in Internet of Things: Taxonomies and open challenges," *Mobile Netw. Appl.*, vol. 24, pp. 796–809, Jun. 2019.

[6] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, Jul. 2017.

[7] M. Devi and A. Majumder, "Side-channel attack in Internet of Things: A survey," in *Proc. Appl. Internet Things (ICCCIOT)*, 2021, pp. 213–222.

[8] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.

[9] I. B. Kraiem, F. Ghozzi, A. Péninou, G. Roman-Jimenez, and O. Teste, "Human-interpretable rules for anomaly detection in time-series," in *Proc. Int. Conf. Extending Database Technol.*, 2021, pp. 457–462.

[10] S. Evmorfos, G. Vlachodimitropoulos, N. Bakalos, and E. Gelenbe, "Neural network architectures for the detection of SYN flood attacks in IoT systems," in *Proc. 13th ACM Int. Conf. Pervasive Technol. Relat. Assist. Environ.*, 2020, pp. 1–4.

[11] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, "Learning graph structures with transformer for multivariate time-series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, Jun. 2022.

[12] F. Abbasi, M. Naderan, and S. E. Alavi, "Anomaly detection in Internet of Things using feature selection and classification based on logistic regression and artificial neural network on N-BaIoT dataset," in *Proc. 5th Int. Conf. Internet Things Appl. (IoT)*, 2021, pp. 1–7.

[13] S. Yaras and M. Dener, "IoT-based intrusion detection system using new hybrid deep learning algorithm," *Electronics*, vol. 13, no. 6, p. 1053, 2024.

[14] M. M. Khan and M. Alkhathami, "Anomaly detection in IoT-based healthcare: Machine learning for enhanced security," *Sci. Rep.*, vol. 14, no. 1, p. 5872, 2024.

[15] A. Abusitta, G. H. S. de Carvalho, O. A. Wahab, T. Halabi, B. C. M. Fung, and S. Al Mamoori, "Deep learning-enabled anomaly detection for IoT systems," *Internet Things*, vol. 21, Apr. 2023, Art. no. 100656.

[16] Y. Liu, H. Wang, X. Zheng, and L. Tian, "An efficient framework for unsupervised anomaly detection over edge-assisted Internet of Things," *ACM Trans. Sens. Netw.*, pp. 1–26, Apr. 2023.

[17] I. Apostol, M. Preda, C. Nila, and I. Bica, "IoT botnet anomaly detection using unsupervised deep learning," *Electronics*, vol. 10, no. 16, p. 1876, 2021.

[18] R.-H. Hwang, M.-C. Peng, C.-W. Huang, P.-C. Lin, and V.-L. Nguyen, "An unsupervised deep learning model for early network traffic anomaly detection," *IEEE Access*, vol. 8, pp. 30387–30399, 2020.

[19] Y. Guo, T. Ji, Q. Wang, L. Yu, G. Min, and P. Li, "Unsupervised anomaly detection in IoT systems for smart cities," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2231–2242, Oct.–Dec. 2020.

[20] R. Bhatia, S. Benno, J. Esteban, T. V. Lakshman, and J. Grogan, "Unsupervised machine learning for network-centric anomaly detection in IoT," in *Proc. 3rd ACM Conext Workshop Big Data, Mach. Learn. Artif. Intell. Data Commun. Netw.*, 2019, pp. 42–48.

[21] N. Lutsiv et al., "Deep Semisupervised learning-based network anomaly detection in heterogeneous information systems," *Comput., Mater. Continua*, vol. 70, no. 1, pp. 414–431, 2022.

[22] S. K. Nukavarapu and T. Nadeem, "Securing edge-based IoT networks with semi-supervised GANs," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. Workshops Affil. Events (PerCom Workshops)*, 2021, pp. 579–584.

[23] S. Liu, X. Hao, and X. Chen, "A semi-supervised dynamic ensemble algorithm for IoT anomaly detection," in *Proc. Int. Conf. Internet Things (iThings) IEEE Green Comput. Commun. (GreenCom) IEEE Cyber, Phys. Soc. Comput. (CPSCom) IEEE Smart Data (SmartData) IEEE Congr. Cybermatics (Cybermatics)*, 2020, pp. 264–269.

[24] Y. Cheng, Y. Xu, H. Zhong, and Y. Liu, "Leveraging semisupervised hierarchical stacking temporal convolutional network for anomaly detection in IoT communication," *IEEE Internet Things J.*, vol. 8, no. 1, pp. 144–155, Jan. 2021.

[25] M. Abdel-Basset, H. Hawash, R. K. Chakrabortty, and M. J. Ryan, "Semi-supervised spatiotemporal deep learning for intrusions detection in IoT networks," *IEEE Internet Things J.*, vol. 8, no. 15, pp. 12251–12265, Aug. 2021.

[26] A. Meyer-Berg, R. Egert, L. Böck, and M. Mühlhäuser, "IoT dataset generation framework for evaluating anomaly detection mechanisms," in *Proc. 15th Int. Conf. Availab., Rel. Security*, 2020, pp. 1–6.

[27] "PyShark: Python wrapper for tshark, the network traffic analysis tool." KimiNewt. Accessed: Oct. 23, 2024. [Online]. Available: https://github.com/KimiNewt/pyshark

[28] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. Int. Conf. Inf. Syst. Security Privacy*, 2017, pp. 253–262.

[29] "CICFlowmeter: Ethernet traffic bi-flow generator and analyzer for anomaly detection." Ahlashkari. Accessed: Oct. 23, 2024. [Online]. Available: https://github.com/ahlashkari/CICFlowMeter

[30] R. Zhao et al., "A novel traffic classifier with attention mechanism for industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 11, pp. 10799–10810, Nov. 2023.

[31] S. A. Bakhsh, M. A. Khan, F. Ahmed, M. S. Alshehri, H. Ali, and J. Ahmad, "Enhancing IoT network security through deep learning-powered intrusion detection system," *Internet Things*, vol. 24, Dec. 2023, Art. no. 100936.

[32] J. Zhao, Q. Li, J. Sun, M. Dong, K. Ota, and M. Shen, "Efficient IoT device identification via network behavior analysis based on time series dictionary," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 5129–5142, Feb. 2024.

[33] A. Behera, K. S. Sahoo, T. K. Mishra, and M. Bhuyan, "A combination learning framework to uncover cyber attacks in IoT networks," *Internet Things*, vol. 28, Dec. 2024, Art. no. 101395.

[34] N. Jeffrey, Q. Tan, and J. R. Villar, "Using ensemble learning for anomaly detection in Cyber–physical systems," *Electronics*, vol. 13, no. 7, p. 1391, 2024.

[35] A. Zohourian, S. Dadkhah, H. Molyneaux, E. C. P. Neto, and A. A. Ghorbani, "IoT-PRIDS: Leveraging packet representations for intrusion detection in IoT networks," *Comput. Security*, vol. 146, Nov. 2024, Art. no. 104034.




**Mahdi Rabbani** received the M.Tech. degree in artificial intelligence from the University of Mysore, Mysore, India, in 2014, and the doctoral degree in computer science and technology from Nanjing University of Science and Technology, Nanjing, China, in 2022.

He is a Postdoctoral Fellow with the Canadian Institute for Cyber Security, University of New Brunswick, Fredericton, NB, Canada. Throughout his career, he has contributed to the field of cybersecurity, with numerous publications in various journals. His research interests encompass AI for cybersecurity, malware analysis, machine learning, and deep-learning-based network anomaly detection and recognition, as well as privacy-enhancing techniques for IoT devices.



**Jinkun Gui** received the Master of Computer Science degree from University of New Brunswick, Fredericton, NB, Canada, in 2024, where he was supervised by Prof. Rongxing Lu.

Beyond academia, he has served as a programming instructor and teaching assistant, fostering the development of future technologists. His research focuses on IoT security, privacy-preserving communication protocols, and intrusion detection systems.

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.



**Fatemeh Nejati** received the B.Sc. degree in mathematics from Shiraz University, Shiraz, Iran, in 2016, and the M.Sc. degree in mathematics from Kharazmi University, Tehran, Iran, in 2019. She is currently pursuing the master's degree in computer science with the University of New Brunswick, Fredericton, NB, Canada.

She serves as a Research Assistant with the Canadian Institute for Cybersecurity, Fredericton. Her research interests include IoT security and malware analysis.



**Zeming Zhou** received the Master of Computer Science degree from the University of New Brunswick, Fredericton, NB, Canada, in 2024.

He conducted research under the supervision of Prof. Rongxing Lu. His research focuses on IoT security, privacy-preserving communication protocols, and intrusion detection systems.



**Arun Kaniyamattam** received the bachelor's degree in biotechnology from Indian Institute of Technology Guwahati, Guwahati, India, in 2013, and the master's degree in computer science from the University of Louisiana at Lafayette, Lafayette, LA, USA, in 2016.

He is a Cybersecurity Software Developer with over five years of experience.

**Mansur Mirani** is the Vice President and a Tech Hub Lead with Mastercard Vancouver Tech Hub, Vancouver, BC, Canada.

**Gunjan Piya** is a Product Manager with Mastercard Vancouver Tech Hub, Vancouver, BC, Canada.

**Igor Opushnyev** is a Principal Software Engineer with Mastercard Vancouver Tech Hub, Vancouver, BC, Canada.



**Rongxing Lu** (Fellow, IEEE) received the Ph.D. degree from the Department of Electrical and Computer Engineering, University of Waterloo, ON, Canada, in 2012.

He was a Postdoctoral Fellow with the University of Waterloo from May 2012 to April 2013. He is currently a Mastercard IoT Research Chair, a University Research Scholar, and a Professor with the Faculty of Computer Science, University of New Brunswick, Fredericton, NB, Canada. Before that, he was an Assistant Professor with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore, from April 2013 to August 2016. His research interests include applied cryptography, privacy enhancing technologies, and IoT-big data security, and privacy.

Prof. Lu also serves as the Chair for IEEE Communications and Information Security Technical Committee, and the Founding Co-Chair for IEEE TEMS Blockchain and Distributed Ledgers Technologies Technical Committee.



**Ali A. Ghorbani** (Senior Member, IEEE) has held a variety of positions in academia for the past 37 years and is currently a Professor of Computer Science, the Tier 1 Canada Research Chair of Cybersecurity, and the Director of the Canadian Institute for Cybersecurity, University of New Brunswick, Fredericton, NB, Canada, which he established in 2016. He served as the Dean of the Faculty of Computer Science, University of New Brunswick from 2008 to 2017 and spent over 27 years of his 37-year academic career. He is also the Founding Director of the Laboratory for Intelligence and Adaptive Systems Research. He is the Co-Founder of the Privacy, Security, Trust Network in Canada, and its international annual conference and the co-inventor of three awarded patents and has published over 260 peer reviewed articles during his career. He has supervised over 170 research associates, postdoctoral fellows, graduates, and undergraduate students during his career. His book, *Intrusion Detection and Prevention Systems: Concepts and Techniques* (Springer, October 2010). His current research focus is cybersecurity, Web intelligence, and critical infrastructure protection.

Mr. Ghorbani is the recipient of the Startup Canada Senior Entrepreneur Award in 2017. He was Twice one of the Three Finalists for the Special Recognition Award at the 2013 and 2016 New Brunswick KIRA Award for the knowledge industry. He served as the Co-Editor-in-Chief for *Computational Intelligence* from 2007 to 2017. He co-founded two startups, Sentrant Security and EyesOver Technologies in 2013 and 2015, respectively.

Authorized licensed use limited to: West Virginia University. Downloaded on October 30,2026 at 12:01:14 UTC from IEEE Xplore. Restrictions apply.